



# SDI Investment Property Marketplace

## System Documentation

What has been built, how to run it, and who can see what

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| Release           | v0.9.0                                                        |
| Repository        | github.com/neilgreene/property-management                     |
| Branch            | main                                                          |
| Commits           | 13                                                            |
| Database          | PostgreSQL 16 (row-level security; requires 15+)              |
| Application       | Node.js 18+ (no framework, no runtime dependencies beyond pg) |
| Automated checks  | 63 worker tests plus 4 SQL walkthroughs, all passing          |
| Deployment status | Local development only. Not deployed, not internet-facing.    |

*Every fact in this document was extracted from the repository and from a freshly built database, not written from memory. Regenerate it with `python3 docs/generate_system_documentation.py`.*

This page intentionally left blank.

# Contents

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>1. What This Is</b>                                 | <b>6</b>  |
| 1.1 The problem it solves first                        | 6         |
| 1.2 Three visibility bands                             | 6         |
| <b>2. How To Run It</b>                                | <b>6</b>  |
| 2.1 Requirements                                       | 6         |
| Route A — Docker (recommended for a first run)         | 6         |
| What is inside the Compose file                        | 7         |
| Route B — local install                                | 7         |
| Optional — only for rebuilding this document           | 7         |
| Network access                                         | 8         |
| 2.2 Docker — nothing installed but Docker              | 8         |
| 2.3 Local PostgreSQL 16+ and Node 18+                  | 8         |
| 2.4 Individual pieces                                  | 8         |
| 2.5 The integration worker                             | 9         |
| <b>3. Users and Access</b>                             | <b>10</b> |
| 3.1 Signing in                                         | 10        |
| 3.2 Seeded people                                      | 10        |
| 3.3 Database roles                                     | 11        |
| <b>4. Architecture</b>                                 | <b>12</b> |
| 4.1 Four schemas, and the separation is load-bearing   | 12        |
| 4.2 Tables                                             | 12        |
| <b>5. The API</b>                                      | <b>12</b> |
| 5.1 Database views — what the application reads        | 12        |
| 5.2 Callable functions                                 | 13        |
| 5.3 HTTP endpoints                                     | 13        |
| <b>6. The GoHighLevel Integration</b>                  | <b>15</b> |
| 6.1 What the worker does                               | 15        |
| 6.2 The fee gate has two conditions, not one           | 15        |
| 6.3 Direction decides what happens to an inbound event | 15        |
| 6.4 Nothing here has ever spoken to GoHighLevel        | 15        |
| What that does and does not buy                        | 15        |
| The assumption register                                | 16        |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| How to close it . . . . .                                                    | 16        |
| <b>7. The Marketplace . . . . .</b>                                          | <b>18</b> |
| 7.1 What the browser can and cannot do . . . . .                             | 18        |
| 7.2 Searching . . . . .                                                      | 18        |
| 7.3 The map, and why gated pins are drawn as rings . . . . .                 | 18        |
| 7.4 The drill-down . . . . .                                                 | 19        |
| 7.5 Plain-English search . . . . .                                           | 19        |
| 7.6 Favourites and saved searches . . . . .                                  | 20        |
| 7.7 Photography . . . . .                                                    | 20        |
| <b>8. Where Listing Data Comes From . . . . .</b>                            | <b>21</b> |
| 8.1 Where the data can actually come from . . . . .                          | 21        |
| 8.2 Four tables and one rule . . . . .                                       | 21        |
| 8.3 Vocabulary is data, not code . . . . .                                   | 22        |
| 8.4 What the nightly job is allowed to do . . . . .                          | 22        |
| 8.5 Why the scraper is a named seam and not an implementation . . . . .      | 22        |
| 8.6 Getting a listing into the system today . . . . .                        | 23        |
| <b>9. Data Rights and Compliance . . . . .</b>                               | <b>24</b> |
| 9.1 Three questions per instrument . . . . .                                 | 24        |
| 9.2 What makes a right actually apply . . . . .                              | 24        |
| 9.3 Advisory first, blocking later . . . . .                                 | 25        |
| 9.4 Fair housing, enforced structurally . . . . .                            | 25        |
| 9.5 The regimes on the register . . . . .                                    | 25        |
| 9.6 Filling the register in . . . . .                                        | 27        |
| <b>10. Getting Properties In . . . . .</b>                                   | <b>28</b> |
| 10.1 Two steps, and why . . . . .                                            | 28        |
| 10.2 Staging holds both the file and our reading of it . . . . .             | 28        |
| 10.3 Two fields deliberately not mapped . . . . .                            | 28        |
| 10.4 Validation blocks, or warns, and the difference is deliberate . . . . . | 29        |
| 10.5 The review screen . . . . .                                             | 29        |
| 10.6 Release records provenance, and says when that is not enough . . . . .  | 29        |
| <b>11. What Is Tested . . . . .</b>                                          | <b>30</b> |
| <b>12. What Is Not Built . . . . .</b>                                       | <b>31</b> |
| <b>13. Next Steps . . . . .</b>                                              | <b>33</b> |

---

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 13.1 What each item needs, and how long . . . . .               | 33        |
| 13.2 Recommended sequence . . . . .                             | 35        |
| 13.3 What must be true before a phase starts . . . . .          | 35        |
| 13.4 How each phase is tested . . . . .                         | 37        |
| The standing regression suite . . . . .                         | 37        |
| Per-phase development tests . . . . .                           | 37        |
| 13.5 Validation methods to consider . . . . .                   | 39        |
| Three rules to attach to whichever of these you adopt . . . . . | 40        |
| 13.6 Deployment . . . . .                                       | 41        |
| Topology . . . . .                                              | 41        |
| <b>14. Where Things Are . . . . .</b>                           | <b>42</b> |
| <b>15. Build and Release . . . . .</b>                          | <b>43</b> |
| 15.1 What gets built . . . . .                                  | 43        |
| 15.2 Cutting a release . . . . .                                | 43        |
| 15.3 What CI produces . . . . .                                 | 43        |
| 15.4 Building locally . . . . .                                 | 44        |
| 15.5 Release checklist . . . . .                                | 44        |
| <b>Appendix A. Table Definitions . . . . .</b>                  | <b>46</b> |
| Schema core . . . . .                                           | 46        |
| Schema feed . . . . .                                           | 52        |
| Schema ghl . . . . .                                            | 55        |
| Schema gov . . . . .                                            | 59        |
| Schema intake . . . . .                                         | 62        |
| <b>Appendix B. Change Log . . . . .</b>                         | <b>66</b> |

## 1. What This Is

A working system for a private investment property marketplace: vetted properties listed with financial analysis, browsable without revealing addresses, with the full detail unlocked only after an investor signs and pays the platform fee agreement. Agents see only their own assignments. Staff see everything.

### 1.1 The problem it solves first

The commercial model depends on one thing holding: an unpaid visitor must not be able to obtain a property's street address. Everything else is ordinary software; that is not. So the visibility rules are enforced **inside the database**, by row-level security policies and column grants, rather than by application code.

This matters because it changes what an attacker has to defeat. If the rule lives in application code, anyone who finds an unguarded query, an API bug, or a direct database connection gets the address. Here, the address is withheld by the database itself: a caller who writes their own SQL, bypasses the application entirely, and connects directly still cannot read it.

### 1.2 Three visibility bands

| Band         | Contains                                                                 | Released to                                              | Enforced by                                                                |
|--------------|--------------------------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------|
| 1 — Public   | City, price, cap rate, NOI, beds, baths, year built                      | Anyone, including anonymous visitors                     | RLS policy per role                                                        |
| 2 — Gated    | Street address, unit, parcel number, seller disclosure, true coordinates | Investors who signed AND paid; the assigned agent; staff | Masking in a <code>security_invoker</code> view, keyed on a data predicate |
| 3 — Internal | Acquisition cost, source channel, staff notes, margin                    | Staff only                                               | Column-level GRANT (a hard ACL)                                            |

Coordinates degrade rather than disappear. An ungated viewer gets a deterministic offset of roughly one kilometre, seeded on the property id, so a map still renders a neighbourhood and repeated page loads cannot be averaged to recover the true point.

## 2. How To Run It

### 2.1 Requirements

Two ways to run this, with different prerequisites. The Docker route needs one thing installed; the local route needs three, but gives faster iteration.

#### Route A — Docker (recommended for a first run)

| Requirement    | Version                                         | Notes                                                                                                    |
|----------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Docker Engine  | 20.10 or later                                  | Any host: Linux, macOS, Windows with WSL2, or a Proxmox VM                                               |
| Docker Compose | v2 (the <code>docker compose</code> subcommand) | Not the older standalone <code>docker-compose</code> binary; this file uses v2 syntax including profiles |

| Requirement           | Version        | Notes                                                                               |
|-----------------------|----------------|-------------------------------------------------------------------------------------|
| Disk                  | about 1.5 GB   | Chiefly the two base images                                                         |
| Memory                | 1 GB free      | PostgreSQL and two small Node processes                                             |
| Network at build time | outbound HTTPS | Pulls images from Docker Hub and pg from the npm registry. Nothing else is fetched. |

## What is inside the Compose file

| Service | Image          | Resolves to                       | Purpose                                                                    | Port |
|---------|----------------|-----------------------------------|----------------------------------------------------------------------------|------|
| db      | postgres:16    | PostgreSQL 16.x — tested on 16.13 | Database; loads every sql/ file on first start                             | 5432 |
| web     | node:22-alpine | Node 22.x — tested on 22.22.2     | The demo interface                                                         | 3000 |
| worker  | node:22-alpine | Node 22.x — tested on 22.22.2     | GoHighLevel integration. Behind the worker profile; not started by default | 3001 |

**Both tags float within their major version, so a pull next month may bring a newer patch release. That is the right default for development — it picks up security fixes — but for production pin them to a digest so a rebuild produces the same image that was tested. The versions above are what this system has actually been verified against.**

Only one runtime dependency is installed at all: pg (resolves to 8.23.x under the declared ^8.13.1). There is no web framework, no ORM, no build step and no bundler. That is deliberate — it keeps the dependency audit trivial and removes a class of supply-chain exposure — and it is worth preserving.

## Route B — local install

| Requirement       | Version             | Why that floor                                                                                                                                                                                     |
|-------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL        | 16 or later         | security_invoker views need 15+; developed and tested on 16. Below 15 a view over an RLS table silently runs as its owner and bypasses the caller's policies, which would defeat the entire model. |
| Node.js           | 18 or later         | Uses the built-in test runner and global fetch, both stable from 18. Tested on 22.                                                                                                                 |
| npm               | 9 or later          | Ships with Node 18+                                                                                                                                                                                |
| psql and createdb | matching the server | run.sh calls them as your own user, which is the default for Postgres.app and Homebrew                                                                                                             |

## Optional — only for rebuilding this document

| Requirement | Version                        | Used by               |
|-------------|--------------------------------|-----------------------|
| Python      | 3.9 or later — tested on 3.11  | Both PDF generators   |
| reportlab   | 4.x or later — tested on 5.0.1 | pip install reportlab |

| Requirement | Version | Used by                                            |
|-------------|---------|----------------------------------------------------|
| pypdfium2   | any     | Optional. Only to render pages for visual checking |

Nothing in the running system needs Python. It is only used to regenerate the two PDFs in docs/.

## Network access

| When                 | Needs                                               | For                                                                          |
|----------------------|-----------------------------------------------------|------------------------------------------------------------------------------|
| Build                | Outbound HTTPS to Docker Hub and registry.npmjs.org | Images and the one npm package                                               |
| Runtime, core system | None                                                | The database, demo and tests are entirely self-contained and run air-gapped  |
| Runtime, worker      | Outbound HTTPS to services.leadconnectorhq.com      | The GoHighLevel API                                                          |
| Runtime, webhooks    | Inbound HTTPS on a public address                   | Only for receiving GoHighLevel deliveries. Everything else works without it. |

The distinction in that last table matters for choosing where to run. The whole system minus webhook receipt works with no inbound access at all, which is why a Proxmox VM with no port forwarding is a perfectly good staging environment.

## 2.2 Docker — nothing installed but Docker

```
git clone https://github.com/neilgreene/property-management
cd property-management
docker compose up
```

Builds the database from sql/, seeds it, and serves the demo at **http://localhost:3000**. PostgreSQL is exposed on localhost:5432 (database sdi, user postgres, password postgres). `docker compose down -v` discards the database.

## 2.3 Local PostgreSQL 16+ and Node 18+

```
./run.sh
```

Loads all eleven schema files, runs all four SQL walkthroughs, runs the 63 worker tests, then starts the demo. It assumes `psql` and `createdb` work as your own user, which is the default for Postgres.app and Homebrew.

## 2.4 Individual pieces

| Command                                          | What it does                                       |
|--------------------------------------------------|----------------------------------------------------|
| <code>psql -d sdi -f sql/05_tests.sql</code>     | Security walkthrough: 11 checks, 5 of them attacks |
| <code>psql -d sdi -f sql/07_ghl_tests.sql</code> | GoHighLevel bridge: 7 checks                       |



| Command                                                   | What it does                                |
|-----------------------------------------------------------|---------------------------------------------|
| <code>psql -d sdi -f<br/>sql/10_review_tests.sql</code>   | Review queue actions: 7 checks              |
| <code>psql -d sdi -f<br/>sql/14_pipeline_tests.sql</code> | Deal visibility and stage history: 9 checks |
| <code>cd worker &amp;&amp; npm test</code>                | 63 unit and end-to-end tests                |

## 2.5 The integration worker

Deliberately not started by a bare `docker compose up`, because it needs real GoHighLevel credentials and there is no point running it without them.

```
GHL_TOKEN=... GHL_LOCATION_ID=... docker compose --profile worker up
```

| Variable                            | Where it comes from                                                            |
|-------------------------------------|--------------------------------------------------------------------------------|
| <code>GHL_TOKEN</code>              | GoHighLevel: Settings → Private Integrations. Scoped to an entire sub-account. |
| <code>GHL_LOCATION_ID</code>        | The sub-account id, visible in the GoHighLevel URL                             |
| <code>GHL_WEBHOOK_PUBLIC_KEY</code> | GoHighLevel's published webhook verification key (PEM)                         |

**The token is read from the environment and is never written into any file in the repository. It grants access to the whole sub-account — contacts, invoices, transactions — so it must never reach browser-side code.**

## 3. Users and Access

### 3.1 Signing in

Authentication is built. A person signs in with an email and a password, gets a session cookie, and the web tier resolves that cookie to a person id and a database role for the duration of one transaction.

| Concern           | How it is handled                                                                                                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Password storage  | scrypt with per-password salt and explicit cost parameters. Node's built-in implementation — no dependency                                                                                                                                         |
| Failed sign-in    | One message and one shape for every failure. Distinguishing “no such account” from “wrong password” is an enumeration oracle, so the failure path does the same hashing work as the success path                                                   |
| Repeated failures | The account locks after five. Setting a new password clears the lock                                                                                                                                                                               |
| Sessions          | A random token; only its SHA-256 is stored, so the session table is not a set of usable credentials. Changing a password revokes every existing session                                                                                            |
| Credential tables | <code>core.credential</code> and <code>core.session</code> are RLS-forced with <b>no policy at all</b> — every direct read is refused for every role, including the owner. They are reachable only through <code>SECURITY DEFINER</code> functions |

**Authentication changed nothing beneath it.** Not one policy, view or grant was modified to add it: the database contract was always “here is a role and an actor id”, and a session now supplies those instead of a dropdown. That is the payoff of having put the authorisation model in the database first.

**The demo persona switcher still exists, but it is off unless `DEMO_PERSONAS=1` is set. A dropdown that hands out an admin session must not be reachable by accident.**

Still missing: password reset, email delivery, and multi-factor. A person who forgets a password needs a staff member to set a new one.

### 3.2 Seeded people

Created by `sql/04_seed.sql`. These are demonstration records, not real people; the addresses and financials attached to them are invented. Every one of them has the password `demo1234`, set by `sql/17_demo_passwords.sql` — a file that belongs in a demo and nowhere else.

| Name         | Role  | Email             | Fee agreement | Brand   | What they demonstrate                             |
|--------------|-------|-------------------|---------------|---------|---------------------------------------------------|
| Jessica Pool | admin | jpool2@yahoo.com  | n/a           | BRAND_A | Full staff access, all bands                      |
| Dan Beitor   | admin | dan@example.com   | n/a           | BRAND_A | Full staff access, all bands                      |
| Tom Bradbury | agent | tom@example.com   | n/a           | BRAND_A | 4 assigned properties, incl. an unpublished draft |
| Priya Raman  | agent | priya@example.com | n/a           | BRAND_A | A second agent, to prove agents are isolated      |

| Name         | Role     | Email              | Fee agreement | Brand   | What they demonstrate                          |
|--------------|----------|--------------------|---------------|---------|------------------------------------------------|
| Ruth Okonkwo | investor | ruth@example.com   | signed        | BRAND_A | Gate open: sees addresses and true coordinates |
| Marcus Pell  | investor | marcus@example.com | not signed    | BRAND_A | Gate shut: same query, address withheld        |
| Ines Duarte  | investor | ines@example.com   | signed        | KAVADOO | The second brand, at concierge pricing         |

Ruth and Marcus are the pair that carries the argument: identical role, identical SQL, and the only difference between them is one timestamp in a data column. The address appears for one and not the other with no application logic involved at all.

### 3.3 Database roles

Application roles are created NOLOGIN and passwordless on purpose. `sdi_app` is NOINHERIT, so it holds no privileges of its own and must assume exactly one persona role per transaction — the application cannot forget to assume a role and accidentally run with more authority than it should.

| Role                         | Purpose                                          | Login | Demo password             |
|------------------------------|--------------------------------------------------|-------|---------------------------|
| <code>sdi_app</code>         | The only role the web tier connects as           | yes   | <code>demo_app_pw</code>  |
| <code>sdi_public</code>      | Anonymous visitors. Band 1 only                  | no    | assumed via SET ROLE      |
| <code>sdi_investor</code>    | Investors. Band 2 if the gate is open            | no    | assumed via SET ROLE      |
| <code>sdi_agent</code>       | Agents. Band 2 on assigned properties            | no    | assumed via SET ROLE      |
| <code>sdi_admin</code>       | Staff. All three bands                           | no    | assumed via SET ROLE      |
| <code>sdi_integration</code> | The GoHighLevel worker. No access to core at all | yes   | <code>demo_int_pw</code>  |
| <code>sdi_test_admin</code>  | Test fixtures only. BYPASSRLS                    | yes   | <code>demo_test_pw</code> |

**Those passwords come from `sql/99_local_logins.sql`, exist so the demo stack is connectable, and are published in a public repository. They are worth exactly nothing and that file must not be loaded anywhere that matters. `sdi_test_admin` carries BYPASSRLS and belongs only in a test database.**

## 4. Architecture

```

browser ■■■ web/server.js ■■■ api.* ■■■ core.* (PostgreSQL)
      sdi_app      views      tables + RLS
      SET ROLE      ■
                  ■■■■ sec.* predicates

GoHighLevel ■■■webhook■■■ worker/src/index.js ■■■ ghl.* (separate schema,
      ■■■REST■■■ sdi_integration                no access to core)

```

### 4.1 Four schemas, and the separation is load-bearing

| Schema | Holds                                               | Who has USAGE                                                                                                                             |
|--------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| core   | Base tables. Properties, people, deals, assignments | No application role. Name resolution fails before any ACL or policy is consulted, so there is no path around the views.                   |
| sec    | Security predicates, definer-rights                 | All persona roles. They live here because a security_invoker view resolves the functions it calls as the caller too, not just the tables. |
| api    | Masking views and the write path                    | All persona roles. The only surface the application reads.                                                                                |
| ghl    | The GoHighLevel bridge                              | sdi_integration only. The web tier cannot enumerate CRM contacts, invoices or transactions.                                               |

Brand is a lens, not a property attribute. Both brands read the same core.property rows; core.property\_brand decides publication and price per brand. Adding the second brand is rows in one table, not a schema refactor.

### 4.2 Tables

| Schema | Tables                                                                                                                           |
|--------|----------------------------------------------------------------------------------------------------------------------------------|
| core   | brand, person, property, property_brand, property_assignment, saved_property, pipeline, pipeline_stage, deal, deal_stage_history |
| ghl    | id_map, webhook_event, transaction, fee_agreement, sync_state, outbox, review_queue, reviewable_field                            |

All ten core tables have row-level security enabled and forced; none of the ghl tables do, because that schema is reachable only by the integration worker and by nothing else.

**Every column of every table is defined in Appendix A**, with its type, nullability, default, key role and foreign key target. That appendix is generated from the live database rather than transcribed, so it cannot describe a schema the system does not actually have.

## 5. The API

### 5.1 Database views — what the application reads

| View                               | Returns                                              | Granted to                     |
|------------------------------------|------------------------------------------------------|--------------------------------|
| <code>api.property</code>          | Properties with band 2 masked or released per caller | public, investor, agent, admin |
| <code>api.property_internal</code> | Band 3: acquisition cost, source, notes, margin      | admin only                     |
| <code>api.my_saved</code>          | The caller's own saved properties                    | investor, admin                |
| <code>api.deal</code>              | Deals the caller is party to, with stage and age     | investor, agent, admin         |
| <code>api.deal_history</code>      | Stage transitions for those deals                    | investor, agent, admin         |
| <code>api.review_open</code>       | Inbound CRM edits awaiting a decision                | admin only                     |

Note that `api.deal` is not granted to `sdi_public` at all. A deal is never public, at any stage.

## 5.2 Callable functions

| Function                                     | Does                                                                              | Caller             |
|----------------------------------------------|-----------------------------------------------------------------------------------|--------------------|
| <code>api.save_property(uuid)</code>         | Saves a property to the caller's list, rejecting anything they cannot see         | investor           |
| <code>api.review_decide(bigint, text)</code> | Accept or reject a queued CRM edit; accepting writes an allowlist of columns only | admin              |
| <code>api.security_invariants()</code>       | Returns zero rows, or names what is broken                                        | admin              |
| <code>ghl.apply_fee_agreement(text)</code>   | Opens the gate for a person, if the document is completed AND paid                | integration worker |

The `sec.*` predicates (`can_see_address`, `can_see_deal`, `is_assigned`, `actor`, `jitter` and others) are internal. They are the shared source of truth that policies, views and write functions all consult, so a rule cannot drift between the place it is read and the place it is enforced.

## 5.3 HTTP endpoints

| Service     | Method and path               | Purpose                                                                     |
|-------------|-------------------------------|-----------------------------------------------------------------------------|
| Marketplace | GET /                         | The marketplace: filters, map, cards, drill-down                            |
| Marketplace | POST /api/login · /api/logout | Sign in and out. One response shape for every failure                       |
| Marketplace | GET /api/whoami               | Who this session is, and whether personas are enabled                       |
| Marketplace | GET /api/listings?<filters>   | Filtered listings, plus facet ranges and the vocabularies for the dropdowns |
| Marketplace | GET /api/property?id=         | One listing in full, with its photographs. 404 when the row policy hides it |
| Marketplace | POST DELETE /api/favorite     | Add or remove a favourite                                                   |
| Marketplace | GET /api/favorites            | The caller's favourites, as full listing cards                              |

| Service     | Method and path                          | Purpose                                                                                           |
|-------------|------------------------------------------|---------------------------------------------------------------------------------------------------|
| Marketplace | GET   POST   DELETE<br>/api/saved-search | List, upsert by name, delete                                                                      |
| Marketplace | POST /api/saved-search/run               | Replay a saved search; returns its criteria and records the run                                   |
| Marketplace | POST /api/parse                          | Plain English to a bounded criteria object — see 7.5                                              |
| Marketplace | GET /media/<id>/<kind>.svg               | Generated listing illustration                                                                    |
| Marketplace | GET /api/view · /api/probe               | The teaching endpoints: one persona's whole payload, and a direct base-table read that is refused |
| Worker      | POST /webhooks/ghl                       | Receives a GoHighLevel delivery, verifies its signature                                           |
| Worker      | GET /healthz                             | Queue depth: pending events, bad signatures, outbox backlog, open reviews                         |

**The marketplace runs on port 3000, the worker on 3001. The marketplace authenticates; the worker does not, and neither should be exposed to a network until the deployment gaps in section 12 are addressed.**

## 6. The GoHighLevel Integration

Built against GoHighLevel's official OpenAPI specifications. The full API contract is documented separately in `docs/GHL - Interface - Specification.pdf` (17 pages).

### 6.1 What the worker does

| Component                         | Responsibility                                                                                                                                      |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ghlClient.js</code>         | HTTP client. Sets the required Version: 2021-07-28 header once, paces under the 100-request/10-second limit, retries 429 and 5xx, never retries 403 |
| <code>signature.js</code>         | Verifies webhook signatures against GoHighLevel's published RSA public key                                                                          |
| <code>webhookReceiver.js</code>   | Records deliveries, deduplicating on <code>webhookId</code> and rejecting stale ones                                                                |
| <code>handlers.js</code>          | Dispatches received events                                                                                                                          |
| <code>outbox.js</code>            | Drains outbound writes with retry that cannot duplicate a record                                                                                    |
| <code>sync/documents.js</code>    | Polls fee agreement state                                                                                                                           |
| <code>sync/transactions.js</code> | Nightly reconciliation of payments                                                                                                                  |
| <code>migrate/</code>             | EspoCRM to GoHighLevel load, in ordered passes                                                                                                      |

### 6.2 The fee gate has two conditions, not one

GoHighLevel reports document status and payment status *independently*. A document can be signed and unpaid. `ghl.apply_fee_agreement()` is the only thing in the system that opens the gate, and it requires both: completed or accepted, **and** paid. A tag-based unlock gets this wrong.

### 6.3 Direction decides what happens to an inbound event

`core.property` is the system of record; GoHighLevel is downstream of it for listings. So an inbound record edit means somebody changed a property inside the CRM, against the grain of the architecture. Applying it would let the CRM silently overwrite the authoritative row. Dropping it would lose a real edit. It is neither: it goes to `ghl.review_queue` for a person to decide.

Events that genuinely originate in GoHighLevel — a signed document, a settled payment, a contact created by staff — are applied directly, because for those GoHighLevel is the source. Accepting a queued edit writes only an allowlist of band 1 columns, so a status change can never become a route to rewriting an address or a cost basis.

### 6.4 Nothing here has ever spoken to GoHighLevel

**This is the most important caveat in the document, and it is easy to miss behind a passing test count. The integration is written and tested, but no line of it has made a request to GoHighLevel. Every test supplies a double: an injected `fetch`, a locally generated RSA keypair, a fake client object. Zero of the 63 tests reach the network.**

### What that does and does not buy

| Verified                                                                                                                                                           | Not verified                                                                                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| The logic: retry and backoff, deduplication, the two-condition fee gate, outbox resumability, migration ordering, allowlist enforcement, every privilege boundary. | Every assumption about how the real API actually behaves: response shapes, field names in practice, error bodies, and the webhook signature algorithm. |
| That the code does the right thing when GoHighLevel responds in the shape the code expects.                                                                        | That GoHighLevel responds in that shape.                                                                                                               |

## The assumption register

Rather than leave that as an unbounded worry, here is the finite list. Each row is a specific assumption the code makes, where it lives, and what happens if it is wrong. Checking these against one real account is a morning's work and closes the whole category.

| # | Assumption                                                       | Where                 | If wrong                                                                          |
|---|------------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------|
| 1 | Signatures are PKCS#1 v1.5 over SHA-256, over the raw body       | signature.js          | Every delivery is rejected as forged. Loud, not silent.                           |
| 2 | Transaction list returns rows under data or transactions         | sync/transaction.s.js | Sync silently reads zero rows and reports success. Quiet, and the worst of these. |
| 3 | Document list returns rows under documents with a total          | sync/documents.js     | The fee gate never opens for anyone. Loud once someone pays.                      |
| 4 | A document's payer is recipients[0].contactId                    | sync/documents.js     | The document cannot be matched to a person; the gate stays shut.                  |
| 5 | Contact upsert returns contact.id or id                          | outbox.js             | The id map is not written, so the association pass cannot resolve.                |
| 6 | Record create returns record.id or id                            | outbox.js             | Same, for properties.                                                             |
| 7 | Relation create takes firstRecordId/secondRecordId/associationId | ghlClient.js          | Relations fail with a 422; the migration reports it rather than losing it.        |
| 8 | Webhook payloads carry type, webhookId and timestamp             | webhookReceiver.js    | Events are recorded as unknown or rejected outright.                              |
| 9 | A 429 carries Retry-After                                        | ghlClient.js          | Backoff falls back to exponential. Harmless.                                      |

**Assumption 2 is the one to check first. Most of these fail loudly — a rejected delivery, a 422, an empty gate somebody complains about. That one fails *quietly*: the sync reports success, the cursor advances, and the ledger simply never fills. A single real response settles it.**

The code is written defensively against exactly this — the fallbacks above are why each row lists two candidate shapes rather than one — which lowers the odds but does not remove them. Guessing carefully is still guessing.

## How to close it



Two artifacts, neither of which needs any code to be written: **one captured webhook delivery** (raw body plus its x-wh-signature header, which worker/tools/capture-webhook.js writes for you) and **one saved response from each of** GET /payments/transactions **and** GET /proposals/document. That is phase P0 in section 9, budgeted at one week, and it is first in the schedule for this reason.

## 7. The Marketplace

The browsing surface an investor actually uses: filters across the top, a map on the left, listing cards on the right, and a drill-down panel over the top. The layout is the one every property portal has converged on, for the reason they converged on it — price and geography are the two questions a buyer asks first, and the layout answers both at once.

### 7.1 What the browser can and cannot do

The front end contains no security logic. It does not decide whose address to show, which photographs to display, or which listings to draw. It draws what the API hands it, and it finds out that an address is withheld at the same moment the reader does — because `street_address` arrives as null.

This is worth stating because it is the difference between a gate and a curtain. A front end that receives every address and hides some of them is defeated by the browser's developer tools. This one is not sent them.

### 7.2 Searching

| Control                         | Filters on                                                       | Where it runs                                      |
|---------------------------------|------------------------------------------------------------------|----------------------------------------------------|
| Price, beds, baths, square feet | Range on each                                                    | Bound parameters against <code>api.property</code> |
| Type, city                      | Exact match; the options are drawn from what this caller can see | Same query                                         |
| Sort                            | Price, cap rate, size, bedrooms                                  | An allowlisted ORDER BY, never interpolated        |
| Plain English                   | Parsed into the same criteria as the controls above              | See 7.5                                            |

Two properties of this that are structural rather than incidental. **A filter can only narrow.** The row policies and the view's masking run first; every filter clause runs inside that result, so there is no filter value that widens what a caller sees. And **filtering happens in the database**, so a listing the caller may not see is never sent and then hidden — it is not sent.

Even the facet ranges — the price floor and ceiling, the bedroom counts offered in the dropdown — are computed from the same policy-bounded relation. The bounds a caller sees are the bounds of their own data, so the filter bar itself does not leak the existence of listings they cannot open.

### 7.3 The map, and why gated pins are drawn as rings

Every listing carries true coordinates in band 2. An ungated viewer receives a deterministic offset of roughly a kilometre instead, seeded on the property id so repeated loads cannot be averaged back to the true point.

The map draws that difference rather than hiding it: an unlocked listing gets a sharp price pin, a gated one gets a soft ring around an approximate centre. Drawing a gated listing as a precise pin would claim an accuracy the data does not have, and would teach the viewer to trust a position that is deliberately wrong.

The map library and its tiles come from a CDN. When that CDN is unreachable — an air-gapped host, a restrictive proxy — a built-in renderer plots the same listings to scale with no dependencies, priced, clickable and hover-linked to their cards, labelled so nobody mistakes it for a basemap. A demo that shows a grey rectangle on a locked-down network is a demo that fails in the room it matters in.

## 7.4 The drill-down

One property, in full. The 404 for a listing the caller may not see is produced by the row policy, not by a check in application code: `api.property_detail` is a view over `api.property`, so an invisible property returns zero rows and the route reports not-found. Not *forbidden* — saying “forbidden” would confirm the listing exists to anyone who guessed an id.

| Section             | Contains                                                                                                                        | Band     |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------|----------|
| Photography         | Interior views. The front elevation is band 2 — see below                                                                       | 1, and 2 |
| Headline figures    | Cap rate, monthly rent, NOI, price per square foot                                                                              | 1        |
| Income and expenses | Gross rent, vacancy, management, tax, insurance, maintenance, utilities, HOA, and the net                                       | 1        |
| Area                | Median household income, median price and rent, rent growth, vacancy, price-to-income, and this rent as a share of local income | 1        |
| The building        | Lot, storeys, garage, parking, heating, cooling, roof year, renovations, features                                               | 1        |
| Location            | Street address, unit, exact coordinates, front elevation photograph                                                             | 2        |

**The whole financial analysis is band 1.** That is a deliberate commercial choice, not an oversight: the analysis is what sells the listing, so withholding it would defeat the purpose of publishing at all. What is withheld is what identifies the parcel — nothing on the page is a teaser.

**Photographs respect the same gate as the address.** A picture of the front of a house identifies it as surely as its street number: a door number, a recognisable streetscape. So `core.property_media` carries `reveals_location`, and any image flagged that way is released on the same predicate as the address itself. Interiors stay public. Without this, the gate would be reopened through the picture gallery.

The expense breakdown sums exactly to the `opex_annual` published on the listing. That is enforced by generation rather than trusted: an investor who adds up the components gets the headline figure back, and a detail page whose arithmetic does not close is a page nobody underwrites on twice.

## 7.5 Plain-English search

A free-text box turns “3 bed duplex in Cleveland under 200k, best yield” into `{min_beds:3, property_type:'Duplex', city:'Cleveland', max_price:200000, sort:'cap_desc'}` and shows the reader what it understood, so the box is never opaque about what it did.

**It is a rules parser today. It is not a language model and it does not call one. It is built now because the shape of the feature — free text in, a bounded criteria object out — is the part that has to be right, and it is worth having that seam built and tested before a model is put behind it.**

The design is what makes a model safe to add later. The parser's only output is a criteria object whose keys are fixed and whose values the query builder binds, and every object passes a validator on the way out. So the blast radius of a wrong answer — from these rules today or from a model tomorrow — is a bad search, never a bad query. Swapping in a model means replacing one function body; the validator is not optional in that world, it is the thing that makes model output executable.

## 7.6 Favourites and saved searches

| Feature        | Stored in                        | Visibility                                                                                                                                                 |
|----------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Favourites     | <code>core.saved_property</code> | Owner only, plus staff. Saving is checked against the same predicate the row policy uses, so an investor cannot favourite a listing their own policy hides |
| Saved searches | <code>core.saved_search</code>   | Owner only, with no staff override. What an investor is hunting for is their own business                                                                  |

Search criteria are stored as `jsonb` because the filter set will keep growing and each addition should not be a migration. The cost of `jsonb` is that it accepts anything — so the storable keys are constrained at the table. A saved search is replayed later, possibly by different code, and what cannot be stored cannot be replayed.

The favourites view joins `api.property`, not `core.property`. An address hidden in the grid must not appear because the listing was favourited; inheriting the mask rather than reimplementing it is what guarantees the two cannot drift apart.

## 7.7 Photography

The listing images are generated: deterministic flat vector illustrations, seeded on the property id so a listing always looks the same. Real photographs are somebody's copyright and a stock service is an account and a dependency, so the demo draws its own and makes them visibly illustrations rather than photo-imitations.

Replacing them is a URL change. `core.property_media` stores a url and nothing outside the renderer assumes where it is served from. The gate lives in the database and does not care where the bytes come from.

## 8. Where Listing Data Comes From

A property in this marketplace is also listed somewhere else — an MLS, a consumer portal, a wholesaler's sheet. It goes under contract, it sells, it is withdrawn, and nobody tells us. An investor who calls about a house that went pending three weeks ago is the single most expensive kind of stale data this system can hold.

**The wrinkle that shapes the entire design: escrow fails.** A property that went pending comes back to market perhaps one time in five. So this cannot be a one-way “mark it gone” job. It has to follow the source in both directions, and a design that only knows how to retire a listing will quietly bury the ones that come back.

### 8.1 Where the data can actually come from

Worth being blunt, because the obvious answer is not available.

| Route                                | What it gives                                                                                                   | What it costs                                                                                                                                               |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MLS via RESO Web API                 | The correct answer. A standardised feed with a closed status vocabulary, served by essentially every modern MLS | An MLS membership or approved vendor status, a signed data agreement, and a per-MLS contract. Coverage is per-market, so a national portfolio means several |
| Bridge Interactive (Zillow Group)    | MLS data under Zillow Group's own platform                                                                      | Same gate: manual approval, restricted to industry partners, brokerages and MLS organisations                                                               |
| Zillow's public API                  | Nothing. The legacy Web API (ZWSID) was retired; there is no self-serve endpoint for live listing inventory     | —                                                                                                                                                           |
| Vendor property APIs (e.g. RentCast) | Self-serve, no membership. Property records, rent comps, and a derived listing status                           | A key and a subscription. Status is derivative rather than primary, which is why it is treated as advisory here                                             |
| Scraping a consumer portal           | Apparent completeness                                                                                           | Outside the portal's terms, and technically the weakest option — see 8.5                                                                                    |
| A person with a browser              | Authoritative, immediate, and available today                                                                   | Someone's time. For a portfolio of a few dozen listings this is a real answer, not a placeholder                                                            |

The system is built so that which of these is used is a row in a table, not a rewrite. Every source implements one adapter interface and every answer goes through the same reconciler, so adding the MLS feed later changes no logic that is already tested.

### 8.2 Four tables and one rule

| Table                               | Holds                                                                                                                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>feed.listing_source</code>    | Who is telling us, and how far they are trusted: authoritative or advisory, whether they may retire a listing, and how many agreeing checks a change needs                                                          |
| <code>feed.property_external</code> | Our property against their key, with <code>last_checked_at</code> and <code>last_seen_at</code> kept apart — “we looked” and “we found it” are different facts, and their difference is how a delisting is detected |

| Table              | Holds                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------|
| feed.observation   | Append-only. What a source said, when, verbatim, alongside our reading of it. Never updated |
| feed.status_change | What we actually did about it, why, and whether a worker or a person did it                 |

**The rule: an observation never writes a status directly.** It is recorded, then reconciled. That separation is what makes the history answerable after the fact — “why is this pending?” has an answer with a timestamp and a source — and what lets an unreliable source raise a flag without being allowed to act.

### 8.3 Vocabulary is data, not code

Every feed has its own words. RESO says *Active Under Contract*; one portal says *Pending*; another says *Contingent* or *Accepting backup offers*. All four mean the same thing here. `feed.status_map` holds the translation, so a term a portal invented last week is a row, not a release.

A term nobody has mapped is **recorded and flagged, never guessed**. An unmapped status appears on an operator worklist as “we saw something we do not understand” rather than silently defaulting to whatever seems closest.

### 8.4 What the nightly job is allowed to do

The sweep runs once a night, walks every enabled watch oldest-check-first so a run cut short has still done the most overdue work, and rate-limits per source. It decides nothing: it records what it saw and the database decides what it means.

| Signal                                                | What happens                                                                                                                                                                          |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The source reports a status                           | Recorded. Acted on once <code>confirm_after</code> checks agree — one reading of a feed is not evidence                                                                               |
| The source reports Active on something we had pending | Acted on <b>immediately</b> . See below                                                                                                                                               |
| The listing is absent from the feed                   | Counted. After enough consecutive absences the listing becomes <i>withdrawn</i> — not <i>sold</i> . Gone is not the same as sold, so we do not guess; withdrawn is the reversible one |
| The request failed                                    | Ignored entirely. An outage is not a market emptying                                                                                                                                  |
| An advisory source disagrees with us                  | A flag on the staff review queue. No status changes                                                                                                                                   |
| A staff member changed the status by hand             | The reconciler defers. A person who set it knows something the feed does not                                                                                                          |

**Two asymmetries, both deliberate.** An error is never read as an absence: an adapter that reports a timeout as “missing” would, over one bad night, walk the entire portfolio to withdrawn, so an adapter in doubt must report an error. And coming back to market is acted on the first sighting rather than the second: a failed escrow is a house that is saleable *now*, and every night it waits shown as unavailable is a night of lost enquiries. The two mistakes do not cost the same, so they are not treated the same.

### 8.5 Why the scraper is a named seam and not an implementation

It is wired in, it is registered as a source, and it returns “not implemented”. The reason is engineering before it is legal.

**A scraper cannot reliably distinguish *this listing is gone* from *the page changed shape*. Both render as a missing selector. So its most common failure mode is indistinguishable from its most destructive signal, and a scraper allowed to retire listings will one day retire the whole portfolio in a single run — correctly, as far as it can tell.**

If one is ever built, two flags stay where they are: it remains *advisory*, so it raises flags instead of changing statuses, and it remains barred from retiring a listing. Those are columns on its source row, already set that way, so switching it on does not silently grant it authority.

## 8.6 Getting a listing into the system today

| Route               | How                                                                                                                                                                                            |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MLS feed            | Set RESO_BASE_URL and RESO_TOKEN, activate the source row. Listings arrive complete and stay current                                                                                           |
| By hand, in bulk    | <code>worker/tools/import-listing.js</code> takes a JSON file of the details and merges it — only the keys present are written, so re-running with one corrected field does not blank the rest |
| By hand, one status | A staff member records what they saw; the next sweep picks it up and it goes through the same reconciler as any feed, so it is auditable and reversible                                        |

The Irvine listing in the seed data (108 Fairgrove, 92618) shows the intended posture. It is tracked — two watches, a real external key — and it is a **draft** with null sizes and zero prices, so it reaches nobody. The facts held are the ones in the source URL; the price, the room counts and the photographs are licensed content that has not been obtained. Inventing plausible numbers to fill the gaps would produce exactly the confident wrong record this schema exists to prevent. It is tracked before it is trusted.

## 9. Data Rights and Compliance

Every listing here is somebody else's data, held under some instrument. Whether it may be shown, to whom, and for how long is not a software question and cannot be answered by looking at the code — but it decides whether a listing may be published, so it is recorded where the publication decision is made: in the database, next to the row.

A licence recorded in a spreadsheet is a licence that gets breached the week somebody forgets the spreadsheet exists.

**None of this is legal advice, and none of it claims to know what any particular agreement says. It is the shape that holds whatever the signed agreements say, plus a register of the regimes identified as applying. Every row carries a review status, and an unreviewed right is visibly unreviewed rather than quietly treated as settled.**

### 9.1 Three questions per instrument

| Question                                   | Recorded as              | Why it decides anything                                                                                                                                                                     |
|--------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Where</b> does it apply?                | gov.data_right_territory | A feed licensed for one market does not cover a property in another. Using it there is a breach even though the software works perfectly — which is exactly why it needs a mechanical check |
| <b>What</b> may be done with it?           | gov.data_right_use       | “Show this to a registered user” is not “publish this to the open web”, and neither is “export it in bulk” or “train a model on it”. Eight uses, each answered separately                   |
| <b>Until</b> when, and in return for what? | gov.obligation           | Attribution wording, refresh cadence, removal within N hours of a delisting, deletion on termination. The ones with deadlines are computed rather than remembered                           |

Two details in that table are doing more work than they look. **Silence is not permission**: a use with no row, or one marked *unclear*, is refused — and the intake tool writes every one of the eight uses out explicitly, so a reader can tell “we never asked” from “they said no”. And **rights are scoped**: a right over listing facts says nothing about the photographs. Listing photography usually belongs to the photographer or the listing broker rather than the seller, and conflating the two is a well-worn way for a portal to get sued.

### 9.2 What makes a right actually apply

Four conditions, and they are AND rather than a score. A right applies only if it is counsel-confirmed, unexpired, covers the property's territory, and grants the specific use being asked about.

gov.may\_use(property\_id, use, scope) is the single question the publication path asks. Every clause in it is a reason the answer is *no*, stated positively, so a future clause cannot accidentally widen it.

A right cannot mark itself confirmed. record-data-right.js only sets that status when a reviewer is named on the command line, because “somebody set a flag in a JSON file” and “a lawyer read the contract” must not look the same afterwards.



### 9.3 Advisory first, blocking later

The business operates today. A control that refused to publish anything without a confirmed right would take a working marketplace off the air over paperwork that has simply not been transcribed yet — and a control like that gets reverted, not fixed.

| Mode               | Behaviour                                                                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| advisory (default) | Publication proceeds and warns. Every uncovered listing appears in <code>gov.uncovered_publication</code> with the reason. The whole gap is visible on day one and nothing breaks |
| blocking           | Publication requires a confirmed right, and the standing invariant fails on any remaining gap. This is the go-live gate, and the invariant is what keeps it shut once flipped     |

Advisory is a recorded decision with a visible consequence, not the absence of one. It sits in `gov.policy` with who changed it and why.

### 9.4 Fair housing, enforced structurally

This is the one part of the register with teeth in the running application, and it deserves the space.

A marketplace that lets a user filter, or an algorithm rank, on a protected characteristic — or on a proxy for one — is steering, whether or not anyone intended it. **The Fair Housing Act does not require intent.** A recommendation engine is as capable of it as a dropdown, which is why the register exists before there is a recommendation engine.

The proxies are the hard part, and they are listed explicitly, because a system that blocks race and permits `percent_white_by_tract` has blocked nothing.

| Registered proxy                 | Proxy for             | Why                                                                                                                                  |
|----------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| School rating                    | Race, national origin | Ratings track catchment demographics. Offering one as a ranking axis is steering by another name                                     |
| Crime index                      | Race, national origin | Reported-crime indices measure policing intensity as much as risk                                                                    |
| Area median income as a ranking  | Race, national origin | The marketplace holds it, and legitimately — it is how rent is estimated. Offering it as an axis a buyer sorts on is a different act |
| Neighbourhood desirability score | Multiple              | A composite is a laundered version of whatever went into it                                                                          |

Three controls, at three levels. The register is a table (`gov.prohibited_dimension`). The standing invariant fails if any listed dimension is ever exposed as a readable column. And **the web tier refuses to start** if its own filter allowlist intersects the register — the check runs against the database rather than living in a comment above the filter array, because the array is what a future feature edits and a rule that exists only in a comment survives exactly until somebody is in a hurry.

**What this does not catch is a dimension added under an innocent name. That is what the register's basis column and code review are for. It catches the careless case, which is the common one.**

### 9.5 The regimes on the register

Identified, with the trigger condition that makes each one apply, and — the column that makes this more than a wall poster — where the constraint is actually enforced. A regime with no control is a gap that is visible rather than assumed.

| Regime                                  | Applies when                                                                                         | Status here                                                                                 |
|-----------------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Fair Housing Act, and state equivalents | Always                                                                                               | Enforced structurally, three ways — see 9.4                                                 |
| RESPA s.8 — referral fees               | If compensated in connection with a federally related mortgage loan, including by referring business | <b>Unresolved and material.</b> See below                                                   |
| State real estate licensing             | Per state where property is marketed                                                                 | Unresolved. Turns on whether the platform's activity is brokerage                           |
| MLS participation, IDX and VOW rules    | From the moment any MLS feed is connected                                                            | Machinery built and unused: attribution, refresh and removal SLAs are modelled and computed |
| Copyright in photographs and copy       | Any media not authored in-house                                                                      | Media provenance tracked separately from listing facts                                      |
| CCPA/CPRA and state privacy statutes    | Per consumer residency and thresholds                                                                | <b>Gap.</b> No subject-request mechanism exists                                             |
| TCPA — SMS and calls                    | Any marketing text or autodialled call, including via GoHighLevel                                    | <b>Gap.</b> Consent is not captured or evidenced here                                       |
| CAN-SPAM                                | Any commercial email                                                                                 | Delegated to GoHighLevel; suppression state is not mirrored locally                         |
| ECOA / Regulation B                     | If credit is applied for, referred or influenced                                                     | Deferred. Applies the day lender matching is built                                          |
| GLBA and the Safeguards Rule            | If the business is a “financial institution”                                                         | Turns on the same facts as RESPA                                                            |
| PCI DSS                                 | Any handling of cardholder data                                                                      | Deferred. No payment integration exists; use a redirected processor when it does            |
| ADA / web accessibility                 | Public-facing web content                                                                            | Unaudited. The map has no non-visual equivalent                                             |
| GDPR                                    | Only if EU or UK data subjects                                                                       | Not applicable today, registered so the trigger stays visible                               |
| Site terms and unauthorised access      | If any portal is read programmatically                                                               | No scraper implemented — see 8.5                                                            |

**RESPA section 8 is the largest open legal question in this product, and it is not a technical one. The model is a \$750 fee that unlocks property information and connects investors to agents and lenders. Whether that is compensation for services actually rendered or an unlawful referral fee turns on the fee's substance rather than its label. It needs counsel before launch, not after — and it should go to counsel together with the state licensing and GLBA questions, because all three turn on the same facts about what the business actually does.**

**TCPA is the highest-frequency risk, because it is the one an ordinary marketing decision can breach, statutory damages are per violation, and consent has to be provable. Nothing in this system currently captures or evidences it.**

## 9.6 Filling the register in

The register currently holds the synthetic demonstration data, a deliberately empty instrument for the externally-tracked Irvine address, and the operator-supplied photograph of it. It does not yet hold the instruments the business actually operates under, because those live in signed paper that has to be transcribed by a person.

`docs/data-rights-intake.md` is the questionnaire for that — five questions per instrument, a worked example, and the loader. `SELECT * FROM api.governance_status` says where things stand at any moment.

| To see                                                        | Query                                                |
|---------------------------------------------------------------|------------------------------------------------------|
| The summary                                                   | <code>SELECT * FROM api.governance_status</code>     |
| Published with no confirmed right                             | <code>SELECT * FROM gov.uncovered_publication</code> |
| The regimes, and which have no control                        | <code>SELECT * FROM api.compliance_register</code>   |
| Per property: what is held, and why it does or does not apply | <code>SELECT * FROM api.data_rights</code>           |

## 10. Getting Properties In

Staff build an analysis workbook per property — one .xlsm holding the offer, the rents, the expenses and a twenty-year projection. Those numbers used to reach the marketplace by being retyped. This is the path that replaces that: load the file into staging, look at what arrived, then release the whole batch or the specific rows that passed review.

### 10.1 Two steps, and why

```
python3 tools/workbook-to-json.py *.xlsm > batch.json
node worker/tools/load-intake.js batch.json --note "August sourcing"
```

Reading .xlsm needs a spreadsheet library, and the worker image has no dependency beyond the Postgres driver. Rather than drag one in, the conversion happens wherever the file already is — a laptop, the host — and what crosses into the database is plain JSON. The middle format is the point: it can be opened and read before anything touches the database.

The reader takes the workbook's Import sheet, by label. It is the only sheet carrying the address, and its 77 labels were identical across the workbooks checked. The flatter One Row export has no address, so it cannot stand alone.

### 10.2 Staging holds both the file and our reading of it

| Table               | Holds                                                                                    |
|---------------------|------------------------------------------------------------------------------------------|
| intake.batch        | One file, one upload, one person, one moment                                             |
| intake.row          | One property. The verbatim payload and the parsed columns, side by side and never merged |
| intake.zip_centroid | The coordinate the workbook does not carry                                               |

**Keeping the raw payload untouched matters more than it looks.** When a released listing turns out to say something surprising, the only useful question is whether the spreadsheet said that or whether we mistranslated it — and that question has no answer if the import overwrote its own input. So it is written once, never edited, and every parsed column beside it is a claim that can be checked against it. The review screen shows it behind a link on every row.

**A workbook has no coordinates, and core.property requires a point. A ZIP centroid is accurate to about a mile, which is exactly what an ungated viewer is shown anyway. A ZIP with no centroid is a blocking error rather than a silent (0,0): a listing quietly pinned to the middle of the wrong city is worse than one that refuses to load.**

### 10.3 Two fields deliberately not mapped

The workbook carries *Schools Rating (scale 3-30)* and a composite FAVORABLE/INSUFFICIENT deal score partly derived from it. Both are already registered in `gov.prohibited_dimension` as fair-housing proxies — see 9.4.

They stay in the raw payload, because the file is not silently edited and staff underwriting may legitimately weigh schools. They never become a column. `api.security_invariants()` fails if either name appears in `core` or `api`.

so this is enforced rather than intended.

#### 10.4 Validation blocks, or warns, and the difference is deliberate

| Level          | Examples                                                                                                                                        | Effect                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>error</b>   | No address, no usable price or rent, no coordinate, an address already listed, the same address twice in one file                               | The row cannot be approved and cannot be released |
| <b>warning</b> | Unusual bedroom count or floor area, expenses meeting or exceeding gross rent, an expense total that does not reconcile with its own components | Shown to the reviewer. Does not block             |

A reviewer forced to clear every oddity before releasing anything stops reading the oddities, and the warnings are then worth nothing. That is the whole reason for the split.

Both copies of a duplicated address are flagged, not just the second. The system cannot know which one was intended, so it refuses them together and leaves the choice to a person.

**The reconciliation check applies the management fee and vacancy allowance to the rent before comparing, rather than summing only tax, insurance and maintenance. An earlier version did the latter and flagged every correctly built workbook — and a check that fires on everything is one a reviewer learns to click past, which costs more than not having it.**

#### 10.5 The review screen

/admin.html, staff only. Batches down the left, rows on the right, with price, rent, NOI and a cap rate computed here rather than trusted from the file.

| Action             | What it does                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Approve selected   | Marks rows reviewed. A row with a blocking error is refused, and the screen says how many of the selected rows actually changed |
| Reject selected    | With an optional reason, kept against the row                                                                                   |
| Release selected   | Creates the listings. Disabled unless something approved is selected, and it names the count                                    |
| what the file said | The verbatim payload for that row                                                                                               |

**Nothing reaches the marketplace without a person releasing it.** An invalid row cannot be approved — approving past a blocking error is how validation stops meaning anything — a pending row cannot be released, and “select all” means “all the releasable ones”, leaving blocked rows exactly where they are. A staging table that auto-promotes on a green validation is not a review queue; it is an import with extra steps.

The screen contains no permission logic. Every action is one of four `api` functions granted to staff alone, so an investor who calls them is refused by the database rather than by an if-statement in the page — which is also why the page settles the question by asking the server for the queue rather than reading a role name out of the session.

#### 10.6 Release records provenance, and says when that is not enough

Releasing writes a `gov.property_provenance` row against the batch's data right. The workbooks are held under `SDI-WORKBOOK`, which is recorded **unreviewed** on purpose:

| What arrives in the file                                      | Whose it is                                                                                                 |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| The financial modelling — offer, rents, expenses, projections | SDI's own work. Needs no external instrument                                                                |
| The property description                                      | Verbatim MLS listing copy, written by the listing agent. The right to republish it has not been established |

Because `gov.may_use()` honours only counsel-confirmed rights, releasing under it publishes with an advisory warning and lists the property in `gov.uncovered_publication`. **The review screen reports that back at the moment of release** rather than leaving it to be found in a report later — the register working on real data rather than demonstration data.

## 11. What Is Tested

Not a coverage percentage. These are the specific claims that are checked, and the attacks that are run against them.

**Read this alongside 6.4. The database suites exercise a real PostgreSQL and prove what they claim. The worker suite proves its logic against test doubles, and proves nothing about GoHighLevel's or an MLS's actual behaviour.**

| Suite                                      | Checks | Notable                                                                                                                                                                         |
|--------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sql/05_tests.sql</code>              | 11     | Five attacks: direct base-table read, another investor's saved list, saving an invisible property, a VOLATILE predicate side-channel, and the standing invariants               |
| <code>sql/07_ghl_tests.sql</code>          | 7      | A signed-but-unpaid document must not open the gate; replay must not move a signature timestamp                                                                                 |
| <code>sql/10_review_tests.sql</code>       | 7      | A payload smuggling an address and a cost basis alongside a legitimate status change; only the allowlist is applied                                                             |
| <code>sql/14_pipeline_tests.sql</code>     | 9      | One agent cannot read another's deal by naming its id; one investor cannot read another's stage history                                                                         |
| <code>sql/23_listing_sync_tests.sql</code> | 10     | The whole listing lifecycle: under contract, failed escrow back to market, a feed outage, a genuine delisting, an advisory source, and a status term nobody has mapped          |
| <code>sql/27_governance_tests.sql</code>   | 10     | A data right built one failing condition at a time, and the same confirmed right refusing to cover a property one state away                                                    |
| <code>sql/29_intake_tests.sql</code>       | 9      | Spreadsheet to listing: an invalid row cannot be approved, a pending row cannot be released, and "release ALL" releases only what was approved                                  |
| <code>web/ (npm test)</code>               | 34     | Password verification cost on the failure path, session revocation on password change, lockout after repeated failures, and that no application role can read a credential hash |

| Suite              | Checks | Notable                                                                                                                                                                                                                                        |
|--------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| worker/ (npm test) | 70     | Signature forgery, replay, oversized bodies, rate-limit handling, ambiguous-create duplication, migration resumability, and the sweep's discipline — an error is never an absence, an advisory source cannot act. All against doubles, see 6.4 |

**Every suite that changes demo data restores it. That is not tidiness: a test that leaves a property pending, or an investor's fee agreement open, breaks the side-by-side contrast the whole demo rests on — and does it silently, one run later.**

`api.security_invariants()` must always return zero rows. It catches the four changes that quietly dismantle the model: USAGE granted on core, an internal column exposed to a non-admin, RLS switched off on a protected table, or a view created without `security_invoker`. Wire it into CI and a nightly check.

## 12. What Is Not Built

Stated plainly so nothing here is mistaken for finished.

| Missing                                | Consequence                                                                                                                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A real listing feed                    | The MLS adapter is written against the RESO standard and has never run against a live feed. Until credentials exist, listing status is maintained by staff. See section 8                 |
| The portal scraper                     | Deliberately unimplemented. The source row, the trust flags and the review queue that would receive its output all exist — see 8.5 for why it stays that way                              |
| Document storage                       | The signed PDF artifact is not stored                                                                                                                                                     |
| Messaging and unified inbox            | Not started                                                                                                                                                                               |
| Audit trail on band 2 and 3 reads      | Who viewed which address, and when, is not recorded                                                                                                                                       |
| Co-investment matching                 | The pipeline exists; the matching engine does not                                                                                                                                         |
| A language model behind the search box | The text parser is rules, not a model. The seam and the validator that would make model output safe to execute are built and tested — see 7.5                                             |
| Real listing photography               | Images are generated illustrations. Swapping them is a URL change; the gate does not depend on where they are served from                                                                 |
| Password reset and email delivery      | Authentication works, but a person who forgets a password needs staff to set a new one                                                                                                    |
| EspoCRM field mapping                  | The load ordering and resumability are built and tested. The field-level mapping needs the live EspoCRM schema                                                                            |
| A hardened deployment                  | The stack runs under Docker Compose and has been deployed on a VM. Nothing is internet-facing, and nothing should be until TLS, a reverse proxy and secret management are in place        |
| The data-rights register, filled in    | The machinery is built and the demo data is covered. The instruments the business actually operates under have not been transcribed — see 9.6 and <code>docs/data-rights-intake.md</code> |

| Missing                                  | Consequence                                                                                                                                                                                                                                                                   |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authorised delivery of gated media       | Real photographs are served as static files under <code>web/public/</code> , so the database controls who is TOLD a url, not who can fetch it. Fine for generated illustrations, not for a location-revealing photograph. Needs an authorising route or signed, expiring urls |
| Consent capture for SMS and email        | TCPA consent is neither captured nor evidenced here, and GoHighLevel's suppression state is not mirrored. This system cannot currently prove an opt-out was honoured                                                                                                          |
| Privacy subject requests                 | No access, deletion or correction mechanism. One implementation satisfies CCPA and most state statutes at once                                                                                                                                                                |
| Uploading a workbook through the browser | The review screen reviews and releases; it does not accept a file. Loading is two commands at a shell — see 10.1                                                                                                                                                              |
| Geocoding                                | Coordinates come from a small ZIP centroid table, accurate to about a mile. A new market needs a row added before its listings will validate                                                                                                                                  |
| Editing a staged row before release      | A wrong figure means correcting the workbook and reloading. There is no in-place edit, deliberately: an edited row no longer matches the payload it is stored beside                                                                                                          |



## 13. Next Steps

Section 12 lists what is missing. This section says what each item needs, in what order, what has to be true before a phase can start, and how each one is tested. Durations are working weeks for one experienced developer and are estimates, not commitments.

### 13.1 What each item needs, and how long

Ordered as recommended in 13.2. Estimates are working weeks for one experienced developer who already knows this codebase, and cover build plus the tests in 13.4. They exclude design iteration, review latency, and waiting on a third party — the three things that actually move dates.

| #  | Item                                        | What it is                                                                                                  | Needs before starting                                                                                                    | Est |
|----|---------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----|
| P0 | Signature verification and first deployment | Confirm how GoHighLevel signs webhooks, and deploy the receiver that checks it.                             | A public HTTPS host and a GoHighLevel account. Nothing else in the system.                                               | 1w  |
| P1 | Authentication and sessions                 | Real sign-in. A session resolves to a person id and a role, which the web tier assumes for the transaction. | A decision on identity provider: own accounts, or an external one. No database change.                                   | 3w  |
| P2 | EspoCRM field mapping and rehearsal         | Which EspoCRM field becomes which column, and which of the 30+ SDI metrics are inputs rather than derived.  | Read access to live EspoCRM or a full export. Load ordering already built and tested.                                    | 4w  |
| P3 | Audit trail on gated reads                  | A record of who saw which address, and when.                                                                | P1. A decision on where band 2 is released from — PostgreSQL cannot trigger on SELECT.                                   | 2w  |
| P4 | Public marketplace UI                       | The real browsing surface: search, filters, property cards, masked map.                                     | P1 for the gated half, P2 for real content, and design direction.                                                        | 4w  |
| P5 | Investor portal and fee flow                | Registration, the fee agreement, and the unlock.                                                            | P4, and a payment provider connected in GoHighLevel. The CRM side is already built.                                      | 3w  |
| P6 | Document storage                            | The signed PDF kept as an artifact, not only as a status flag.                                              | A storage and retention decision. The signed document is a financial record.                                             | 2w  |
| P7 | Agent portal                                | An agent's own assignments and conversations, and nothing else.                                             | P1. Per-agent isolation is already enforced and tested at the database level.                                            | 2w  |
| P8 | Messaging and unified inbox                 | Email, SMS and WhatsApp threaded against a contact and a property.                                          | A decision on whether GoHighLevel owns the conversation or only relays it. That sets the data model, so settle it first. | 3w  |
| P9 | External status feed                        | Nightly check that a listing has not gone pending or sold elsewhere.                                        | A source. A licensed feed is strongly preferable to scraping a portal that forbids it.                                   | 2w  |

| #       | Item                      | What it is                                                           | Needs before starting                                                                 | Est<br>. |
|---------|---------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------|
| P1<br>0 | Co-investment<br>matching | Pairing two vetted investors on<br>one property.                     | The matching rules, in writing. The<br>COINVEST pipeline and stages already<br>exist. | 3w       |
| P1<br>1 | Hardening and<br>cutover  | Production configuration,<br>backups, restore rehearsal,<br>go-live. | Everything above that is in scope for launch.                                         | 2w       |

Total build effort is roughly **31 developer-weeks**. The calendar in 13.2 is 20 weeks because P2, P7 and P9 run alongside other work rather than after it. With one developer and no parallelism the same scope is about 31 weeks; the difference is entirely whether the EspoCRM and operations tracks can proceed independently.

### 13.2 Recommended sequence

Two things drive this order. **Authentication gates everything user-facing**, so it goes first and almost nothing can be demonstrated to a real user before it lands. And **the audit trail should precede real investor data**, not follow it — the first time anyone asks who saw an address, the answer has to already exist.

P0 is deliberately tiny and first. It deploys nothing but a webhook receiver, which authenticates by signature rather than by session, so it needs no login and exposes no data. It settles the one unverified item in the system and proves the deployment path at the same time.

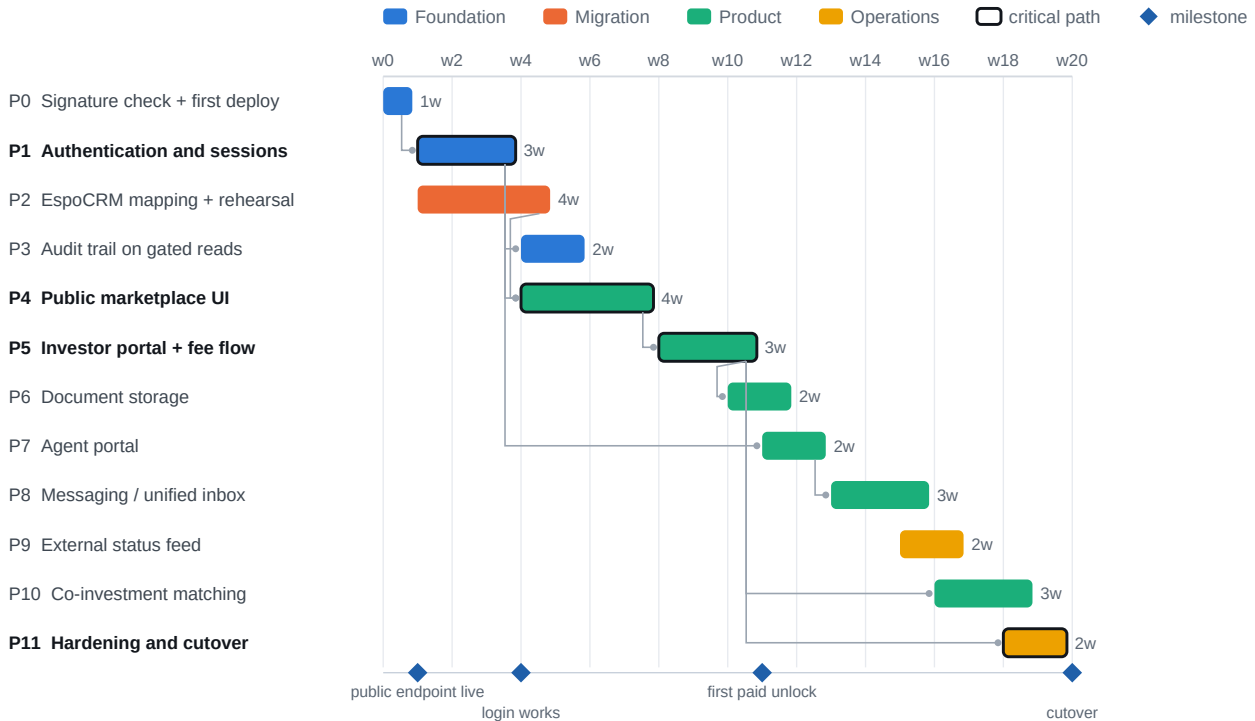


Figure 1. Indicative schedule, one developer. P2 runs alongside P1 because the migration work needs EspoCRM access rather than the new authentication, so the two do not contend.

The critical path is P1 → P4 → P5: authentication, then the browsing surface, then the paid unlock. Everything else can move without moving the launch date. If the schedule has to compress, P8, P9 and P10 are the ones to defer — none of them is required for an investor to find a property, pay, and see the address.

### 13.3 What must be true before a phase starts

| Phase | Entry condition                                                       | Done when                                                                                                           |
|-------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| P0    | A host with a public HTTPS address and a GoHighLevel account          | A live delivery verifies against the published key, and the algorithm is confirmed in code and in the spec document |
| P1    | A decision on the identity provider: own accounts, or an external one | A session resolves to a person id and role; every existing SQL walkthrough still passes unchanged                   |

| Phase | Entry condition                                                       | Done when                                                                                                       |
|-------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| P2    | Read access to live EspoCRM, plus the field mapping agreed in writing | A full rehearsal load into a disposable sub-account reconciles with zero shortfall and zero unresolved links    |
| P3    | P1 complete. A decision on where band 2 is released from              | Every band 2 and band 3 read is attributable to a person and a time; the attack tests still pass                |
| P4    | P1 complete; design direction; listing content from P2                | An anonymous visitor can browse and filter, and cannot obtain an address by any route including the network tab |
| P5    | P4 complete; a payment provider connected in GoHighLevel              | An investor signs, pays, and the address unlocks — driven end to end against a GoHighLevel test sub-account     |
| P6    | A storage and retention decision                                      | The signed PDF is retrievable and its retention is enforced                                                     |
| P7    | P1 complete                                                           | An agent sees only their own assignments and conversations, proven by test, not by inspection                   |
| P8    | The ownership decision in 13.1                                        | A thread is readable against both the contact and the property                                                  |
| P9    | A data source                                                         | A status change reaches the review queue and no listing changes without a human                                 |
| P10   | Matching rules in writing                                             | Two vetted investors are matched and both are notified                                                          |
| P11   | Everything above that is in scope for launch                          | Cutover rehearsed on staging, with a tested rollback                                                            |

### 13.4 How each phase is tested

Every phase adds its own tests and must leave the existing ones green. That second half is the part that usually erodes, so it is stated as a gate rather than a habit.

#### The standing regression suite

| Suite                     | Checks | Must remain                                           |
|---------------------------|--------|-------------------------------------------------------|
| sql/05_tests.sql          | 11     | All five attacks refused                              |
| sql/07_ghl_tests.sql      | 7      | Signed-but-unpaid still does not open the gate        |
| sql/10_review_tests.sql   | 7      | The allowlist still refuses band 2 and band 3 columns |
| sql/14_pipeline_tests.sql | 9      | No cross-agent or cross-investor read                 |
| worker/ npm test          | 63     | All passing                                           |
| api.security_invariants() | —      | Zero rows, on every build                             |

**Run the whole thing with `./run.sh`, which loads the schema, runs all four walkthroughs and the worker suite in one pass. Wire `api.security_invariants()` into CI and into a nightly job: it catches the four changes that quietly dismantle the model, and it catches them whether they came from a migration, a hotfix, or a well-meant grant.**

#### Per-phase development tests

| Phase | New tests it must bring                                                                                        |
|-------|----------------------------------------------------------------------------------------------------------------|
| P0    | Signature verification against a captured live delivery, added as a fixture so it is checked forever after     |
| P1    | Session to role mapping; an expired session; a tampered session; a session for a deactivated person            |
| P2    | Reconciliation counts; a resumed load after a forced failure; a link whose endpoints are missing               |
| P3    | An audit row exists for every band 2 and band 3 release; the audit log cannot be written by the reader         |
| P4    | An anonymous HTTP response contains no restricted field — asserted on the response body, not the rendered page |
| P5    | Signed-and-paid unlocks; signed-and-unpaid does not; a refund after unlock                                     |
| P6    | The stored artifact matches what was signed; retention removes it on schedule                                  |
| P7    | One agent cannot reach another's assignment or conversation by id                                              |
| P8    | A message is attributable to a contact and a property; no cross-tenant leak                                    |
| P9    | A source change raises exactly one review item; a flapping source does not raise many                          |
| P10   | A match requires two vetted parties; an unvetted party is never matched                                        |
| P11   | A restore from backup produces a working system; rollback returns to the prior version                         |

Two habits worth keeping, both learned building what already exists. Write the test that tries the attack, not only the one that proves the feature — several of the checks in the suite exist because the naive version of a test passed while

proving nothing. And re-run the whole suite, not the file you are working on: two of the bugs found so far only appeared when suites ran together.

### 13.5 Validation methods to consider

Section 13.4 says what to test. This says *how*, and which technique earns its place where. Not all of these are worth adopting; they are listed with the judgement attached rather than as a checklist to complete.

| Method                                      | What it is good for here                                                                                                                                                                                         | Worth it?                                                                          |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Integration tests against a real PostgreSQL | Row-level security, policies and grants cannot be mocked — a mock will happily return rows the real database would refuse. The existing suite already works this way.                                            | Already in use. Keep it; never substitute mocks for the database.                  |
| Adversarial tests                           | A test that tries the attack, not one that proves the feature. Every 'ATTACK' check in the SQL walkthroughs is one, and several exist because the naive version passed while proving nothing.                    | Essential. One per privilege boundary, minimum.                                    |
| Property-based testing                      | Generate random (viewer, property, entitlement) triples and assert the invariant: a viewer without a settled agreement never receives a street address. This explores combinations nobody thought to write down. | High value from P3 onward. The visibility model is an unusually good fit.          |
| Contract testing against the OpenAPI specs  | Validate outbound request shapes against GoHighLevel's published specification, so a field rename is caught at build time rather than as a 422 in production.                                                    | Worth it once P2 volume is real.                                                   |
| Fault injection                             | Deliberately fail mid-operation: kill the worker between an API call and its database write, redeliver a webhook, duplicate an outbox row. The resumability claims are only true if they survive this.           | Essential before P11. Cheap to run, and the failure it catches is data corruption. |
| Load testing                                | The GoHighLevel burst limit is 100 requests per 10 seconds shared across all callers. Confirm the cached read model absorbs concurrent browsing rather than passing it through.                                  | Before P4 goes public. Model realistic concurrency, not a synthetic peak.          |
| Penetration testing                         | An independent attempt to obtain a street address without paying. The commercial model rests on that being impossible, which makes it the one thing worth an outside opinion.                                    | Before go-live. Scope it explicitly at the gate.                                   |
| Migration reconciliation                    | Counts and spot checks between source and destination after every rehearsal, with unresolved links reported rather than dropped.                                                                                 | Already built. Run it on every rehearsal, not just the final one.                  |
| User acceptance testing                     | Walk the seeded personas through real tasks with real staff. Ruth and Marcus exist precisely so the difference is demonstrable to a non-technical observer.                                                      | Per phase, from P4.                                                                |
| Accessibility testing                       | The public marketplace is a consumer surface. Keyboard navigation, contrast and screen reader behaviour on listing cards and filters.                                                                            | During P4, not after. Retrofitting is far more expensive.                          |

| Method                               | What it is good for here                                                                                                                                            | Worth it?                                                                 |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Data quality checks                  | Standing assertions on the ledger: no negative amounts, no refund exceeding its charge, no transaction pointing at a missing invoice. Some are already constraints. | Promote to constraints where possible — a constraint cannot be forgotten. |
| Static analysis and dependency audit | Lint, type checking, and a scan of the dependency tree. This project has one runtime dependency, which keeps this cheap.                                            | In CI from now. The cost is near zero while the tree is small.            |

### Three rules to attach to whichever of these you adopt

- **Run the whole suite, not the file you are working on.** Two of the bugs found while building this only appeared when suites ran together — one because two test files shared a database and raced.
- **A test that cannot fail is not a test.** Two checks here originally passed for the wrong reason: a permission error fired before the code under test was ever reached. Make each new test fail once, deliberately, before trusting it.
- **Wire `api.security_invariants()` into CI and a nightly job.** It is the only check that catches a policy quietly dismantled by a later migration, and it costs one query.

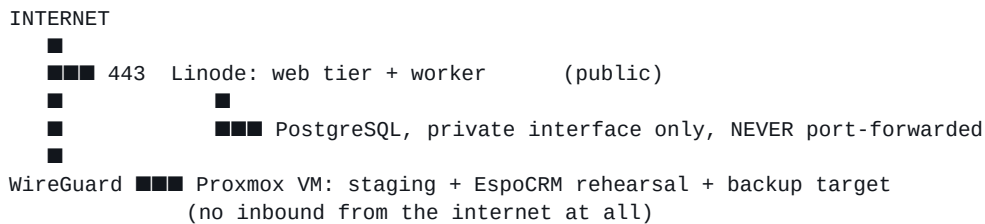


## 13.6 Deployment

Both available options are suitable, for different jobs. The recommendation is to use both rather than choose.

| Environment        | Role                                                         | Why                                                                                                                                                                                                                                                               |
|--------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Proxmox VM (local) | Development and staging. The full stack, no public exposure. | The EspoCRM rehearsal belongs here: real client data never leaves the network, and a rehearsal against a snapshot is repeatable in a way a live read is not. A VM rather than a container — PostgreSQL wants a stable filesystem and its own kernel-level tuning. |
| Linode (public)    | Production. Web tier and worker, publicly reachable.         | P0 needs a public HTTPS endpoint for GoHighLevel to deliver to, and nothing else does until P4. Start with the smallest instance that runs the worker.                                                                                                            |

## Topology



Four rules for the production environment, each of which is cheap now and expensive to retrofit:

- **PostgreSQL is never exposed to the internet.** Bind it to the private interface. The proxy this project was built behind refuses raw database ports for exactly this reason.
- **sdi\_test\_admin must not exist in production.** It carries BYPASSRLS and exists only for test fixtures. `worker/test/bootstrap.sql` is not a deployment script.
- **sql/99\_local\_logins.sql must not be loaded in production.** Its passwords are published in a public repository. Production roles get credentials from the deployment, not from the repository.
- **The GoHighLevel token lives in the environment,** never in a file and never in browser-reachable code. It is scoped to an entire sub-account.

**Backups are the one thing to set up before there is anything worth backing up, because that is the only time it is easy. Nightly pg\_dump to the Proxmox side over WireGuard, and a restore rehearsed at least once — an untested backup is a belief, not a backup.**

## 14. Where Things Are

| Path                           | Contents                                                                                                                                                                                                                                      |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sql/01-04                      | Schema, RLS policies, masking views, demo data                                                                                                                                                                                                |
| sql/05, 07, 10, 14, 23, 27, 29 | The seven walkthroughs. Each is readable top to bottom as an argument, and each restores what it changes                                                                                                                                      |
| sql/06, 08, 09                 | GoHighLevel bridge, review queue, review actions                                                                                                                                                                                              |
| sql/11-13                      | Deals, stage history, pipeline policies and seed                                                                                                                                                                                              |
| sql/15_auth.sql                | Credentials and sessions — see 3.1                                                                                                                                                                                                            |
| sql/16-17, 20                  | The demo dataset: 24 listings, their passwords, and the generated detail and market data                                                                                                                                                      |
| sql/18-19                      | Property detail, market areas, photographs; saved searches                                                                                                                                                                                    |
| sql/21-22                      | Listing sources, status vocabulary, reconciliation — see section 8                                                                                                                                                                            |
| sql/24-26                      | Data rights, territories, permitted uses, the compliance register, the fair-housing prohibited list — see section 9                                                                                                                           |
| sql/28                         | Spreadsheet intake: staging, validation, review and release — see section 10                                                                                                                                                                  |
| tools/                         | The workbook reader. Python, because it needs a spreadsheet library the worker deliberately does not                                                                                                                                          |
| sql/99_local_logins.sql        | Demo passwords. Local development only                                                                                                                                                                                                        |
| web/                           | The marketplace. <code>server.js</code> , <code>auth.js</code> , <code>nlq.js</code> (the text parser), <code>media.js</code> (the illustrations), and <code>public/</code> (the marketplace and <code>admin.html</code> , the review screen) |
| web/test/                      | 34 tests                                                                                                                                                                                                                                      |
| worker/src/                    | The GoHighLevel worker and the listing sweep ( <code>listings/</code> )                                                                                                                                                                       |
| worker/test/                   | 69 tests                                                                                                                                                                                                                                      |
| worker/tools/                  | Webhook capture, the nightly sweep, the listing importer, the data-rights loader and the intake loader                                                                                                                                        |
| docs/data-rights-intake.md     | The questionnaire that turns signed agreements into enforceable rows                                                                                                                                                                          |
| docs/                          | This document and the GoHighLevel interface specification, with their generators                                                                                                                                                              |
| README.md                      | The same ground in more technical detail, including the reasoning behind each design decision                                                                                                                                                 |

*There is no `INSTALL` file; section 2 of this document and the `README`'s opening section cover installation. Both `PDFs` in `docs/` are generated by committed scripts rather than written by hand, so they can be regenerated as the system changes and cannot quietly drift out of date.*

## 15. Build and Release

How the three container images are produced, how a release is cut, and what a deployment actually pulls. The short version: nothing is built on a host. Every image is built by CI from a tagged commit and pulled by tag — which is what makes the stack deployable to Swarm, which cannot build at all, and keeps build tooling off the machine serving traffic.

### 15.1 What gets built

| Image  | Base           | Built from           | Contains                                                                                                                                   |
|--------|----------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| db     | postgres:16    | docker/db.Dockerfile | Every schema file, copied into /docker-entrypoint-initdb.d/, plus the role-login script. The test walkthroughs are deliberately NOT copied |
| web    | node:22-alpine | web/Dockerfile       | server.js, auth.js, media.js, nlq.js and public/                                                                                           |
| worker | node:22-alpine | worker/Dockerfile    | src/ and tools/ — the operator entry points have to be inside the image, not merely in the repository                                      |

One runtime dependency across both Node images: pg. Installed with `npm ci` rather than `npm install`, so the build is reproducible and fails loudly if the lockfile and the manifest disagree. Both run as the unprivileged node user.

**The web image names each module it copies rather than globbing. That is not fussiness: an earlier version listed only two of the four, which produced an image that passed every test and crashed on its first require — a break invisible outside a container build.**

### 15.2 Cutting a release

```
# 1. update the release level and its entry
echo 0.10.0 > VERSION
$EDITOR CHANGELOG.md
python3 docs/extract_schema.py          # if the schema changed
python3 docs/generate_system_documentation.py
python3 docs/generate_test_plan.py

# 2. commit, tag, push
git commit -am "Release 0.10.0"
git tag -a v0.10.0 -m "0.10.0"
git push && git push --tags
```

The push builds three images; the tag builds them again carrying the version. VERSION and CHANGELOG.md are read by this document at generation time, so the release level on the cover and the change log in Appendix B cannot disagree with the repository.

### 15.3 What CI produces

| Tag    | When                                     | Use it for         |
|--------|------------------------------------------|--------------------|
| latest | Every push to main or the working branch | The demo. It moves |

| Tag            | When               | Use it for                        |
|----------------|--------------------|-----------------------------------|
| v0.10.0, 0.10  | A v* tag           | Anything real. Pin it             |
| sha-<full sha> | Every build        | Reproducing an exact deployment   |
| Branch name    | Every branch build | Testing a branch before it merges |

`docker-compose.release.yml` reads `SDI_VERSION` and defaults to `latest`. Set it to a release tag for anything that matters:

```
SDI_VERSION=v0.9.0 docker compose -f docker-compose.release.yml up -d
```

**The workflow pins `latest` to two named branches rather than using the `is_default_branch` template. This repository's default branch is a working branch, so that template evaluated false and the first release published only an immutable `sha-` tag — leaving nothing for a compose file referencing `latest` to pull, which looked exactly like a permissions problem and was not.**

## 15.4 Building locally

Only needed to test a Dockerfile change; the deployment never does this.

```
docker compose build                # all three, from source
docker compose up -d

./db-rebuild.sh                    # database only, no Docker
(cd web && npm test) && (cd worker && npm test)
```

| Script                              | Does                                                                                   |
|-------------------------------------|----------------------------------------------------------------------------------------|
| <code>run.sh</code>                 | Loads every schema file into a local PostgreSQL, then runs all seven walkthroughs      |
| <code>db-rebuild.sh</code>          | Drops and reloads the database, including the test fixture role that is easy to forget |
| <code>docs/extract_schema.py</code> | Snapshots the live schema to <code>docs/schema-snapshot.json</code> for Appendix A     |

**Adding a schema file means editing three lists, and missing one produces a failure far from its cause: `run.sh` for local development, `db-rebuild.sh` for a fast rebuild, and `docker/db.Dockerfile` for the image. Test walkthroughs go in the first two only.**

## 15.5 Release checklist

| # | Check                                   | How                                                                                              |
|---|-----------------------------------------|--------------------------------------------------------------------------------------------------|
| 1 | Every test passes                       | <code>npm test</code> in <code>web/</code> and <code>worker/</code> ; the seven SQL walkthroughs |
| 2 | No standing invariant is violated       | <code>SELECT * FROM api.security_invariants()</code> returns zero rows                           |
| 3 | A clean rebuild works                   | <code>./db-rebuild.sh</code> from nothing, not an incremental patch                              |
| 4 | The demo fixture is intact              | Marcus's gate shut, Ruth's open. A test suite that leaves it otherwise has broken the demo       |
| 5 | New schema files are in all three lists | See 15.4                                                                                         |
| 6 | The schema snapshot is current          | <code>python3 docs/extract_schema.py</code>                                                      |
| 7 | Both PDFs regenerated                   | They read <code>VERSION</code> and <code>CHANGELOG.md</code> at generation time                  |

| # | Check                           | How                          |
|---|---------------------------------|------------------------------|
| 8 | The change log entry is written | CHANGELOG.md, before the tag |

## Appendix A. Table Definitions

Every column of every base table, read from the live database by `docs/extract_schema.py` and stored in `docs/schema-snapshot.json`. Regenerate the snapshot after any schema change and rebuild this document; the appendix cannot then describe a schema the system does not have.

| Marker   | Meaning                                                    |
|----------|------------------------------------------------------------|
| PK       | Part of the primary key                                    |
| FK       | Foreign key; the referenced table is named in the same row |
| UQ       | Covered by a unique constraint                             |
| CK       | Covered by a check constraint                              |
| not null | The column is NOT NULL                                     |

### Schema core

#### core.brand

4 columns · row-level security enforced

| Column       | Type          | Null     | Key | Default / references |
|--------------|---------------|----------|-----|----------------------|
| brand_code   | text          | not null | PK  |                      |
| display_name | text          | not null |     |                      |
| service_tier | text          | not null | CK  |                      |
| platform_fee | numeric(10,2) | not null |     |                      |

#### core.credential

6 columns · row-level security enforced · No RLS policy exists on purpose. Direct access is refused for every role; `api.authenticate()` and `api.set_password()` are the only paths.

| Column          | Type        | Null     | Key   | Default / references |
|-----------------|-------------|----------|-------|----------------------|
| person_id       | uuid        | not null | FK PK | → core.person        |
| password_hash   | text        | not null |       |                      |
| failed_attempts | integer     | not null |       | 0                    |
| locked_until    | timestamptz |          |       |                      |
| password_set_at | timestamptz | not null |       | now()                |
| updated_at      | timestamptz | not null |       | now()                |

#### core.deal

13 columns · row-level security enforced

| Column        | Type          | Null     | Key | Default / references  |
|---------------|---------------|----------|-----|-----------------------|
| deal_id       | uuid          | not null | PK  | gen_random_uuid()     |
| external_ref  | text          |          | UQ  |                       |
| property_id   | uuid          | not null | FK  | → core.property       |
| investor_id   | uuid          |          | FK  | → core.person         |
| agent_id      | uuid          |          | FK  | → core.person         |
| pipeline_code | text          | not null | FK  | → core.pipeline_stage |
| stage_code    | text          | not null | FK  | → core.pipeline_stage |
| amount        | numeric(12,2) |          |     |                       |
| opened_at     | timestamptz   | not null | CK  | now()                 |
| closed_at     | timestamptz   |          | CK  |                       |
| lost_reason   | text          |          |     |                       |
| created_at    | timestamptz   | not null |     | now()                 |
| updated_at    | timestamptz   | not null |     | now()                 |

### core.deal\_stage\_history

7 columns · row-level security enforced · Append-only, written by trigger. The application cannot forget to log a transition, and cannot rewrite one after the fact.

| Column          | Type        | Null     | Key | Default / references |
|-----------------|-------------|----------|-----|----------------------|
| id              | bigint      | not null | PK  | auto-increment       |
| deal_id         | uuid        | not null | FK  | → core.deal          |
| from_stage      | text        |          |     |                      |
| to_stage        | text        | not null |     |                      |
| changed_at      | timestamptz | not null |     | now()                |
| changed_by      | uuid        |          |     |                      |
| seconds_in_from | bigint      |          |     |                      |

### core.market\_area

11 columns · row-level security enforced

| Column  | Type | Null     | Key | Default / references |
|---------|------|----------|-----|----------------------|
| area_id | text | not null | PK  |                      |
| city    | text | not null | UQ  |                      |

| Column                  | Type          | Null     | Key | Default / references |
|-------------------------|---------------|----------|-----|----------------------|
| state                   | char(2)       | not null | UQ  |                      |
| median_household_income | numeric(12,2) |          |     |                      |
| median_home_price       | numeric(12,2) |          |     |                      |
| median_rent_monthly     | numeric(10,2) |          |     |                      |
| rent_growth_1y_bps      | integer       |          |     |                      |
| vacancy_rate_bps        | integer       |          |     |                      |
| population              | integer       |          |     |                      |
| price_to_income         | numeric(5,2)  |          |     |                      |
| as_of                   | date          | not null |     | CURRENT_DATE         |

**core.person***7 columns · row-level security enforced*

| Column                  | Type        | Null     | Key | Default / references |
|-------------------------|-------------|----------|-----|----------------------|
| person_id               | uuid        | not null | PK  | gen_random_uuid()    |
| role                    | actor_role  | not null |     |                      |
| full_name               | text        | not null |     |                      |
| email                   | text        | not null | UQ  |                      |
| fee_agreement_signed_at | timestamptz |          |     |                      |
| home_brand              | text        |          | FK  | → core.brand         |
| active                  | boolean     | not null |     | true                 |

**core.pipeline***4 columns · row-level security enforced*

| Column        | Type    | Null     | Key | Default / references |
|---------------|---------|----------|-----|----------------------|
| pipeline_code | text    | not null | PK  |                      |
| display_name  | text    | not null |     |                      |
| brand_code    | text    |          | FK  | → core.brand         |
| active        | boolean | not null |     | true                 |

**core.pipeline\_stage***6 columns · row-level security enforced*



| Column        | Type    | Null     | Key         | Default / references |
|---------------|---------|----------|-------------|----------------------|
| pipeline_code | text    | not null | FK PK<br>UQ | → core.pipeline      |
| stage_code    | text    | not null | PK          |                      |
| display_name  | text    | not null |             |                      |
| position      | integer | not null | UQ          |                      |
| is_won        | boolean | not null | CK          | false                |
| is_lost       | boolean | not null | CK          | false                |

## core.property

27 columns · row-level security enforced

| Column            | Type          | Null     | Key | Default / references |
|-------------------|---------------|----------|-----|----------------------|
| property_id       | uuid          | not null | PK  | gen_random_uuid()    |
| listing_ref       | text          | not null | UQ  |                      |
| status            | text          | not null | CK  |                      |
| city              | text          | not null |     |                      |
| state             | char(2)       | not null |     |                      |
| zip               | text          | not null |     |                      |
| property_type     | text          | not null |     |                      |
| beds              | smallint      |          |     |                      |
| baths             | numeric(3,1)  |          |     |                      |
| sqft              | integer       |          |     |                      |
| year_built        | smallint      |          |     |                      |
| list_price        | numeric(12,2) | not null |     |                      |
| gross_rent_annual | numeric(12,2) | not null |     |                      |
| opex_annual       | numeric(12,2) | not null |     |                      |
| hoa_annual        | numeric(12,2) | not null |     | 0                    |
| noi_annual        | numeric(12,2) |          |     |                      |
| cap_rate          | numeric(6,4)  |          |     |                      |
| street_address    | text          | not null |     |                      |
| unit              | text          |          |     |                      |

| Column            | Type          | Null     | Key | Default / references |
|-------------------|---------------|----------|-----|----------------------|
| lat               | numeric(9,6)  | not null |     |                      |
| lng               | numeric(9,6)  | not null |     |                      |
| parcel_number     | text          |          |     |                      |
| seller_disclosure | text          |          |     |                      |
| acquisition_cost  | numeric(12,2) |          |     |                      |
| source_channel    | text          |          |     |                      |
| internal_notes    | text          |          |     |                      |
| created_at        | timestampz    | not null |     | now()                |

### core.property\_assignment

3 columns · row-level security enforced

| Column      | Type | Null     | Key   | Default / references |
|-------------|------|----------|-------|----------------------|
| property_id | uuid | not null | FK PK | → core.property      |
| person_id   | uuid | not null | FK PK | → core.person        |
| assign_role | text | not null | CK PK |                      |

### core.property\_brand

4 columns · row-level security enforced

| Column      | Type          | Null     | Key   | Default / references |
|-------------|---------------|----------|-------|----------------------|
| property_id | uuid          | not null | FK PK | → core.property      |
| brand_code  | text          | not null | FK PK | → core.brand         |
| published   | boolean       | not null |       | false                |
| brand_price | numeric(12,2) |          |       |                      |

### core.property\_detail

22 columns · row-level security enforced

| Column              | Type          | Null     | Key   | Default / references |
|---------------------|---------------|----------|-------|----------------------|
| property_id         | uuid          | not null | FK PK | → core.property      |
| headline            | text          |          |       |                      |
| description         | text          |          |       |                      |
| market_rent_monthly | numeric(10,2) |          |       |                      |
| rent_basis          | text          |          | CK    |                      |

| Column                | Type          | Null     | Key | Default / references |
|-----------------------|---------------|----------|-----|----------------------|
| property_tax_annual   | numeric(10,2) |          |     |                      |
| insurance_annual      | numeric(10,2) |          |     |                      |
| utilities_monthly     | numeric(10,2) |          |     |                      |
| utilities_paid_by     | text          |          | CK  |                      |
| maintenance_annual    | numeric(10,2) |          |     |                      |
| management_fee_bps    | integer       |          |     |                      |
| vacancy_allowance_bps | integer       |          |     |                      |
| lot_sqft              | integer       |          |     |                      |
| stories               | smallint      |          |     |                      |
| garage_spaces         | smallint      |          |     |                      |
| heating               | text          |          |     |                      |
| cooling               | text          |          |     |                      |
| roof_year             | smallint      |          |     |                      |
| last_renovated        | smallint      |          |     |                      |
| parking               | text          |          |     |                      |
| features              | jsonb         | not null |     | '[]'                 |
| updated_at            | timestamptz   | not null |     | now()                |

### core.property\_media

7 columns · row-level security enforced

| Column           | Type    | Null     | Key | Default / references |
|------------------|---------|----------|-----|----------------------|
| media_id         | uuid    | not null | PK  | gen_random_uuid()    |
| property_id      | uuid    | not null | FK  | → core.property      |
| url              | text    | not null |     |                      |
| caption          | text    |          |     |                      |
| position         | integer | not null |     | 0                    |
| is_primary       | boolean | not null |     | false                |
| reveals_location | boolean | not null |     | false                |

### core.saved\_property

3 columns · row-level security enforced

| Column      | Type       | Null     | Key   | Default / references |
|-------------|------------|----------|-------|----------------------|
| person_id   | uuid       | not null | FK PK | → core.person        |
| property_id | uuid       | not null | FK PK | → core.property      |
| saved_at    | timestampz | not null |       | now()                |

### core.saved\_search

7 columns · row-level security enforced

| Column      | Type       | Null     | Key   | Default / references |
|-------------|------------|----------|-------|----------------------|
| search_id   | uuid       | not null | PK    | gen_random_uuid()    |
| person_id   | uuid       | not null | FK UQ | → core.person        |
| name        | text       | not null | CK UQ |                      |
| criteria    | jsonb      | not null | CK    | '{}'                 |
| created_at  | timestampz | not null |       | now()                |
| last_run_at | timestampz |          |       |                      |
| run_count   | integer    | not null |       | 0                    |

### core.session

8 columns · row-level security enforced

| Column       | Type       | Null     | Key | Default / references |
|--------------|------------|----------|-----|----------------------|
| token_hash   | bytea      | not null | PK  |                      |
| person_id    | uuid       | not null | FK  | → core.person        |
| created_at   | timestampz | not null |     | now()                |
| expires_at   | timestampz | not null |     |                      |
| last_seen_at | timestampz | not null |     | now()                |
| revoked_at   | timestampz |          |     |                      |
| user_agent   | text       |          |     |                      |
| ip           | inet       |          |     |                      |

## Schema feed

### feed.listing\_source

11 columns · no row-level security

| Column        | Type       | Null     | Key | Default / references |
|---------------|------------|----------|-----|----------------------|
| source_code   | text       | not null | PK  |                      |
| name          | text       | not null |     |                      |
| kind          | text       | not null | CK  |                      |
| base_url      | text       |          |     |                      |
| authoritative | boolean    | not null |     | false                |
| confirm_after | smallint   | not null | CK  | 2                    |
| may_retire    | boolean    | not null |     | false                |
| active        | boolean    | not null |     | true                 |
| check_cron    | text       |          |     |                      |
| notes         | text       |          |     |                      |
| created_at    | timestampz | not null |     | now()                |

## feed.manual\_check

8 columns · no row-level security

| Column      | Type          | Null     | Key | Default / references |
|-------------|---------------|----------|-----|----------------------|
| check_id    | bigint        | not null | PK  | auto-increment       |
| property_id | uuid          | not null | FK  | → core.property      |
| raw_status  | text          | not null |     |                      |
| list_price  | numeric(12,2) |          |     |                      |
| note        | text          |          |     |                      |
| noted_by    | uuid          |          | FK  | → core.person        |
| noted_at    | timestampz    | not null |     | now()                |
| consumed_at | timestampz    |          |     |                      |

## feed.observation

11 columns · no row-level security

| Column         | Type   | Null     | Key | Default / references  |
|----------------|--------|----------|-----|-----------------------|
| observation_id | bigint | not null | PK  | auto-increment        |
| run_id         | uuid   |          |     |                       |
| property_id    | uuid   | not null | FK  | → core.property       |
| source_code    | text   | not null | FK  | → feed.listing_source |

| Column        | Type          | Null     | Key | Default / references |
|---------------|---------------|----------|-----|----------------------|
| observed_at   | timestampz    | not null |     | now()                |
| outcome       | text          | not null | CK  |                      |
| raw_status    | text          |          |     |                      |
| mapped_status | text          |          |     |                      |
| list_price    | numeric(12,2) |          |     |                      |
| payload       | jsonb         |          |     |                      |
| error_detail  | text          |          |     |                      |

## feed.property\_external

12 columns · no row-level security

| Column          | Type          | Null     | Key         | Default / references  |
|-----------------|---------------|----------|-------------|-----------------------|
| property_id     | uuid          | not null | FK PK       | → core.property       |
| source_code     | text          | not null | FK PK<br>UQ | → feed.listing_source |
| external_id     | text          | not null | UQ          |                       |
| external_url    | text          |          |             |                       |
| first_seen_at   | timestampz    | not null |             | now()                 |
| last_checked_at | timestampz    |          |             |                       |
| last_seen_at    | timestampz    |          |             |                       |
| last_raw_status | text          |          |             |                       |
| last_price      | numeric(12,2) |          |             |                       |
| miss_streak     | smallint      | not null |             | 0                     |
| agree_streak    | smallint      | not null |             | 0                     |
| enabled         | boolean       | not null |             | true                  |

## feed.review\_flag

10 columns · no row-level security

| Column      | Type   | Null     | Key | Default / references  |
|-------------|--------|----------|-----|-----------------------|
| flag_id     | bigint | not null | PK  | auto-increment        |
| property_id | uuid   | not null | FK  | → core.property       |
| source_code | text   |          | FK  | → feed.listing_source |

| Column         | Type        | Null     | Key | Default / references |
|----------------|-------------|----------|-----|----------------------|
| observation_id | bigint      |          | FK  | → feed.observation   |
| kind           | text        | not null |     |                      |
| detail         | text        | not null |     |                      |
| raised_at      | timestamptz | not null |     | now()                |
| resolved_at    | timestamptz |          |     |                      |
| resolved_by    | uuid        |          | FK  | → core.person        |
| resolution     | text        |          |     |                      |

### feed.status\_change

9 columns · no row-level security

| Column         | Type        | Null     | Key | Default / references  |
|----------------|-------------|----------|-----|-----------------------|
| change_id      | bigint      | not null | PK  | auto-increment        |
| property_id    | uuid        | not null | FK  | → core.property       |
| from_status    | text        | not null |     |                       |
| to_status      | text        | not null |     |                       |
| reason         | text        | not null |     |                       |
| source_code    | text        |          | FK  | → feed.listing_source |
| observation_id | bigint      |          | FK  | → feed.observation    |
| actor          | text        | not null |     | 'worker'              |
| changed_at     | timestamptz | not null |     | now()                 |

### feed.status\_map

3 columns · no row-level security

| Column        | Type | Null     | Key   | Default / references  |
|---------------|------|----------|-------|-----------------------|
| source_code   | text | not null | FK PK | → feed.listing_source |
| raw_status    | text | not null | PK    |                       |
| mapped_status | text |          | CK    |                       |

## Schema gh1

### gh1.fee\_agreement

10 columns · no row-level security

| Column         | Type          | Null     | Key | Default / references |
|----------------|---------------|----------|-----|----------------------|
| document_id    | text          | not null | PK  |                      |
| location_id    | text          | not null |     |                      |
| person_id      | uuid          |          | FK  | → core.person        |
| ghl_contact_id | text          |          |     |                      |
| status         | text          | not null |     |                      |
| payment_status | text          | not null |     |                      |
| grand_total    | numeric(12,2) |          |     |                      |
| ghl_updated_at | timestamptz   | not null |     |                      |
| observed_at    | timestamptz   | not null |     | now()                |
| raw            | jsonb         | not null |     |                      |

### ghl.id\_map

6 columns · no row-level security

| Column      | Type        | Null     | Key   | Default / references |
|-------------|-------------|----------|-------|----------------------|
| entity_type | text        | not null | PK    |                      |
| local_id    | uuid        | not null | PK    |                      |
| ghl_id      | text        | not null | UQ    |                      |
| ghl_object  | object_kind | not null |       |                      |
| location_id | text        | not null | PK UQ |                      |
| synced_at   | timestamptz | not null |       | now()                |

### ghl.outbox

13 columns · no row-level security

| Column          | Type         | Null     | Key | Default / references |
|-----------------|--------------|----------|-----|----------------------|
| id              | bigint       | not null | PK  | auto-increment       |
| idempotency_key | text         | not null | UQ  |                      |
| operation       | text         | not null |     |                      |
| entity_type     | text         |          |     |                      |
| local_id        | uuid         |          |     |                      |
| payload         | jsonb        | not null |     |                      |
| state           | outbox_state | not null |     | 'pending'            |



| Column          | Type        | Null     | Key | Default / references |
|-----------------|-------------|----------|-----|----------------------|
| attempts        | integer     | not null |     | 0                    |
| next_attempt_at | timestamptz | not null |     | now()                |
| last_error      | text        |          |     |                      |
| ghl_id          | text        |          |     |                      |
| created_at      | timestamptz | not null |     | now()                |
| sent_at         | timestamptz |          |     |                      |

### ghl.review\_queue

12 columns · no row-level security

| Column     | Type         | Null     | Key | Default / references |
|------------|--------------|----------|-----|----------------------|
| id         | bigint       | not null | PK  | auto-increment       |
| source     | text         | not null |     |                      |
| event_type | text         | not null |     |                      |
| ghl_object | text         |          |     |                      |
| ghl_id     | text         |          |     |                      |
| local_id   | uuid         |          |     |                      |
| summary    | text         | not null |     |                      |
| proposed   | jsonb        | not null |     |                      |
| state      | review_state | not null |     | 'open'               |
| raised_at  | timestamptz  | not null |     | now()                |
| decided_at | timestamptz  |          |     |                      |
| decided_by | uuid         |          |     |                      |

### ghl.reviewable\_field

1 columns · no row-level security · Allowlist. An accepted CRM edit can write these and nothing else. Deliberately excludes every band 2 and band 3 column: a status change must not be a route to rewriting an address or a cost basis.

| Column      | Type | Null     | Key | Default / references |
|-------------|------|----------|-----|----------------------|
| column_name | text | not null | PK  |                      |

### ghl.sync\_state

7 columns · no row-level security

| Column       | Type        | Null     | Key | Default / references |
|--------------|-------------|----------|-----|----------------------|
| resource     | text        | not null | PK  |                      |
| location_id  | text        | not null |     |                      |
| cursor_at    | timestamptz |          |     |                      |
| last_run_at  | timestamptz |          |     |                      |
| last_ok_at   | timestamptz |          |     |                      |
| last_error   | text        |          |     |                      |
| records_seen | bigint      | not null |     | 0                    |

## ghl.transaction

17 columns · no row-level security

| Column           | Type          | Null     | Key | Default / references |
|------------------|---------------|----------|-----|----------------------|
| ghl_id           | text          | not null | PK  |                      |
| location_id      | text          | not null |     |                      |
| contact_id       | text          |          |     |                      |
| invoice_id       | text          |          |     |                      |
| subscription_id  | text          |          |     |                      |
| amount           | numeric(12,2) | not null | CK  |                      |
| currency         | char(3)       | not null |     |                      |
| amount_refunded  | numeric(12,2) | not null | CK  | 0                    |
| status           | text          | not null |     |                      |
| live_mode        | boolean       | not null |     |                      |
| payment_provider | text          |          |     |                      |
| entity_type      | text          |          |     |                      |
| entity_id        | text          |          |     |                      |
| ghl_created_at   | timestamptz   | not null |     |                      |
| ghl_updated_at   | timestamptz   | not null |     |                      |
| synced_at        | timestamptz   | not null |     | now()                |
| raw              | jsonb         | not null |     |                      |

## ghl.webhook\_event

9 columns · no row-level security

| Column       | Type       | Null     | Key | Default / references |
|--------------|------------|----------|-----|----------------------|
| webhook_id   | text       | not null | PK  |                      |
| event_type   | text       | not null |     |                      |
| occurred_at  | timestampz | not null |     |                      |
| received_at  | timestampz | not null |     | now()                |
| processed_at | timestampz |          |     |                      |
| attempts     | integer    | not null |     | 0                    |
| last_error   | text       |          |     |                      |
| signature_ok | boolean    | not null |     |                      |
| payload      | jsonb      | not null |     |                      |

## Schema gov

### gov.data\_right

15 columns · no row-level security

| Column               | Type       | Null     | Key | Default / references  |
|----------------------|------------|----------|-----|-----------------------|
| right_id             | text       | not null | PK  |                       |
| name                 | text       | not null |     |                       |
| grantor              | text       | not null |     |                       |
| instrument           | text       | not null | CK  |                       |
| source_code          | text       |          | FK  | → feed.listing_source |
| reference            | text       |          |     |                       |
| counterparty_contact | text       |          |     |                       |
| effective_from       | date       |          |     |                       |
| effective_to         | date       |          |     |                       |
| survives_termination | boolean    | not null |     | false                 |
| review_status        | text       | not null | CK  | 'unreviewed'          |
| reviewed_by          | text       |          |     |                       |
| reviewed_on          | date       |          |     |                       |
| notes                | text       |          |     |                       |
| created_at           | timestampz | not null |     | now()                 |

**gov.data\_right\_territory***2 columns · no row-level security*

| Column       | Type | Null     | Key   | Default / references |
|--------------|------|----------|-------|----------------------|
| right_id     | text | not null | FK PK | → gov.data_right     |
| territory_id | text | not null | FK PK | → gov.territory      |

**gov.data\_right\_use***4 columns · no row-level security*

| Column    | Type | Null     | Key   | Default / references |
|-----------|------|----------|-------|----------------------|
| right_id  | text | not null | FK PK | → gov.data_right     |
| use_code  | text | not null | FK PK | → gov.use_kind       |
| posture   | text | not null | CK    | 'unclear'            |
| condition | text |          |       |                      |

**gov.obligation***7 columns · no row-level security*

| Column         | Type    | Null     | Key | Default / references |
|----------------|---------|----------|-----|----------------------|
| obligation_id  | bigint  | not null | PK  | auto-increment       |
| right_id       | text    | not null | FK  | → gov.data_right     |
| kind           | text    | not null | CK  |                      |
| interval_hours | integer |          |     |                      |
| text_required  | text    |          |     |                      |
| detail         | text    | not null |     |                      |
| enforcement    | text    | not null | CK  | 'unenforced'         |

**gov.policy***5 columns · no row-level security*

| Column           | Type       | Null     | Key   | Default / references |
|------------------|------------|----------|-------|----------------------|
| id               | boolean    | not null | CK PK | true                 |
| enforcement_mode | text       | not null | CK    | 'advisory'           |
| changed_at       | timestampz | not null |       | now()                |
| changed_by       | text       |          |       |                      |
| note             | text       |          |       |                      |

**gov.prohibited\_dimension***4 columns · no row-level security*

| Column    | Type | Null     | Key | Default / references |
|-----------|------|----------|-----|----------------------|
| dimension | text | not null | PK  |                      |
| basis     | text | not null |     |                      |
| kind      | text | not null | CK  |                      |
| rationale | text | not null |     |                      |

**gov.property\_provenance***4 columns · no row-level security*

| Column      | Type        | Null     | Key   | Default / references |
|-------------|-------------|----------|-------|----------------------|
| property_id | uuid        | not null | FK PK | → core.property      |
| right_id    | text        | not null | FK PK | → gov.data_right     |
| scope       | text        | not null | CK PK | 'listing_facts'      |
| acquired_at | timestamptz | not null |       | now()                |

**gov.regulation***9 columns · no row-level security*

| Column       | Type | Null     | Key | Default / references |
|--------------|------|----------|-----|----------------------|
| reg_code     | text | not null | PK  |                      |
| name         | text | not null |     |                      |
| citation     | text |          |     |                      |
| regime       | text | not null | CK  |                      |
| applies_when | text | not null |     |                      |
| constrains   | text | not null |     |                      |
| our_posture  | text | not null |     |                      |
| status       | text | not null | CK  | 'identified'         |
| reviewed_on  | date |          |     |                      |

**gov.regulation\_control***4 columns · no row-level security*

| Column   | Type | Null     | Key   | Default / references |
|----------|------|----------|-------|----------------------|
| reg_code | text | not null | FK PK | → gov.regulation     |

| Column     | Type | Null     | Key | Default / references |
|------------|------|----------|-----|----------------------|
| control    | text | not null | PK  |                      |
| located_in | text | not null |     |                      |
| kind       | text | not null | CK  |                      |

## gov.territory

7 columns · no row-level security

| Column       | Type    | Null     | Key | Default / references |
|--------------|---------|----------|-----|----------------------|
| territory_id | text    | not null | PK  |                      |
| kind         | text    | not null | CK  |                      |
| name         | text    | not null |     |                      |
| country      | char(2) | not null |     | 'US'                 |
| state        | char(2) |          | CK  |                      |
| parent_id    | text    |          | FK  | → gov.territory      |
| notes        | text    |          |     |                      |

## gov.territory\_place

3 columns · no row-level security

| Column       | Type    | Null     | Key   | Default / references |
|--------------|---------|----------|-------|----------------------|
| territory_id | text    | not null | FK PK | → gov.territory      |
| city         | text    | not null | PK    |                      |
| state        | char(2) | not null | PK    |                      |

## gov.use\_kind

3 columns · no row-level security

| Column   | Type | Null     | Key | Default / references |
|----------|------|----------|-----|----------------------|
| use_code | text | not null | PK  |                      |
| label    | text | not null |     |                      |
| detail   | text | not null |     |                      |

## Schema intake

### intake.batch

9 columns · no row-level security

| Column          | Type       | Null     | Key | Default / references |
|-----------------|------------|----------|-----|----------------------|
| batch_id        | uuid       | not null | PK  | gen_random_uuid()    |
| source_file     | text       | not null |     |                      |
| source_kind     | text       | not null | CK  | 'sdi_workbook'       |
| mapping_version | text       | not null |     | 'v1'                 |
| right_id        | text       |          | FK  | → gov.data_right     |
| uploaded_by     | uuid       |          | FK  | → core.person        |
| uploaded_at     | timestampz | not null |     | now()                |
| note            | text       |          |     |                      |
| status          | text       | not null | CK  | 'open'               |

## intake.row

37 columns · no row-level security

| Column         | Type          | Null     | Key   | Default / references |
|----------------|---------------|----------|-------|----------------------|
| row_id         | uuid          | not null | PK    | gen_random_uuid()    |
| batch_id       | uuid          | not null | FK UQ | → intake.batch       |
| row_number     | integer       | not null | UQ    |                      |
| raw            | jsonb         | not null |       |                      |
| street_address | text          |          |       |                      |
| unit           | text          |          |       |                      |
| city           | text          |          |       |                      |
| state          | char(2)       |          |       |                      |
| zip            | text          |          |       |                      |
| property_type  | text          |          |       |                      |
| beds           | smallint      |          |       |                      |
| baths          | numeric(3,1)  |          |       |                      |
| sqft           | integer       |          |       |                      |
| year_built     | smallint      |          |       |                      |
| lat            | numeric(9,6)  |          |       |                      |
| lng            | numeric(9,6)  |          |       |                      |
| list_price     | numeric(12,2) |          |       |                      |

| Column                | Type          | Null     | Key | Default / references |
|-----------------------|---------------|----------|-----|----------------------|
| gross_rent_annual     | numeric(12,2) |          |     |                      |
| opex_annual           | numeric(12,2) |          |     |                      |
| hoa_annual            | numeric(12,2) |          |     |                      |
| market_rent_monthly   | numeric(10,2) |          |     |                      |
| property_tax_annual   | numeric(10,2) |          |     |                      |
| insurance_annual      | numeric(10,2) |          |     |                      |
| maintenance_annual    | numeric(10,2) |          |     |                      |
| management_fee_bps    | integer       |          |     |                      |
| vacancy_allowance_bps | integer       |          |     |                      |
| lot_sqft              | integer       |          |     |                      |
| garage_spaces         | smallint      |          |     |                      |
| description           | text          |          |     |                      |
| internal_notes        | text          |          |     |                      |
| status                | text          | not null | CK  | 'pending'            |
| problems              | jsonb         | not null |     | []                   |
| reviewed_by           | uuid          |          | FK  | → core.person        |
| reviewed_at           | timestampz    |          |     |                      |
| review_note           | text          |          |     |                      |
| property_id           | uuid          |          | FK  | → core.property      |
| released_at           | timestampz    |          |     |                      |

## intake.zip\_centroid

6 columns · no row-level security · Approximate ZIP centroids. Replace with a real geocoder before the address gate opens on anything that matters: once unlocked, the pin is only as accurate as this table.

| Column | Type         | Null     | Key | Default / references |
|--------|--------------|----------|-----|----------------------|
| zip    | text         | not null | PK  |                      |
| city   | text         | not null |     |                      |
| state  | char(2)      | not null |     |                      |
| lat    | numeric(9,6) | not null |     |                      |
| lng    | numeric(9,6) | not null |     |                      |



| Column | Type | Null     | Key | Default / references |
|--------|------|----------|-----|----------------------|
| source | text | not null |     | 'approximate'        |

## Appendix B. Change Log

Read from CHANGELOG.md at generation time, so this document cannot describe a release history the repository does not have. The current release level is v0.9.0, from VERSION.

**Versions are 0.x deliberately: nothing here has run against a production GoHighLevel account or a live MLS feed, so the interfaces are not stable enough to promise compatibility. The first release that has is 1.0.0.**

### 0.9.0

2026-09-02

#### Spreadsheet intake, the review queue, and a test plan.

##### Added

- Schema intake: staged property loads. `intake.batch` (one file, one upload), `intake.row` (one property, holding both the verbatim payload and our reading of it), `intake.zip_centroid`.
- `tools/workbook-to-json.py` — reads the SDI analysis workbook's Import sheet by label and emits intake JSON. Python, because it needs a spreadsheet library the worker deliberately does not carry.
- `worker/tools/load-intake.js` — loads that JSON, validates every row, and prints what a reviewer has to decide on.
- `/admin.html` — the staff review screen. Approve, reject and release, by row or by batch, with the verbatim payload one click away.
- `api.review_intake_rows()`, `api.approve_batch()`, `api.release_intake_rows()`, `api.release_batch()`; views `api.intake_batch` and `api.intake_row`.
- `docs/Feature-Test-Plan.pdf` — 23 pages of acceptance tests, including a section whose every test is expected to be refused.
- System documentation section 10, *Getting Properties In*.

##### Changed

- Validation splits blocking errors from warnings. A reviewer forced to clear every oddity before releasing anything stops reading the oddities.
- Release records provenance against `gov.data_right` SDI-WORKBOOK, recorded **unreviewed**: the financial modelling is SDI's own, but the property description in the same file is verbatim MLS listing copy whose republication right is not established.

##### Fixed

- `api.release_intake_rows()` did not close a batch it had emptied, so a batch released row by row read "open" forever.
- The review screen's select-all box kept its checked state across the redraw that follows every action while the selection was cleared, so the next click unselected everything.
- Worker integration tests proved the fee gate *opens* and never shut it again, leaving Marcus's gate open and silently destroying the demo's central contrast on every test run. Now restores the fixture and asserts the restore worked.

##### Security

- The workbook's "Schools Rating" and its composite deal score are kept in the raw payload and never promoted to a column. Both are registered fair-housing proxies; `api.security_invariants()` fails if either name appears in `core` or `api`.

## 0.8.0

2026-09-02

### Data rights, territory, and regulatory compliance.

#### Added

- Schema gov: `data_right`, `data_right_territory`, `data_right_use`, `obligation`, `property_provenance`, `territory`, `regulation`, `regulation_control`, `prohibited_dimension`, `policy`.
- `gov.may_use(property_id, use, scope)` — the single question the publication path asks. Confirmed, unexpired, in territory, and granting this use: all four, not a score.
- Register of 17 regulatory regimes with the trigger condition for each and where the constraint is actually enforced. Ten currently have no control and say so.
- `docs/data-rights-intake.md` and `worker/tools/record-data-right.js`.
- System documentation section 9, *Data Rights and Compliance*.

#### Changed

- Governance ships in **advisory** mode. A control that refused to publish without a confirmed right would take a working marketplace off the air over paperwork nobody has transcribed yet. Flipping `gov.policy` to `blocking` is the go-live gate; the standing invariant then keeps it shut.
- `api.security_invariants()` gained three governance checks.

#### Security

- Fair housing enforced three ways: the prohibited-dimension register is a table, the standing invariant fails if any listed dimension is exposed as a readable column, and the web tier refuses to start if its filter allowlist intersects the register.

## 0.7.0

2026-09-01

### Listing status tracked against an external source.

#### Added

- Schema feed: `listing_source`, `property_external`, `observation`, `status_change`, `status_map`, `review_flag`, `manual_check`.
- `feed.observe()` and `feed.reconcile()`. An observation never writes a status directly; it is recorded, then reconciled.
- Four source adapters behind one interface: manual, RESO Web API, RentCast, and a consumer portal left deliberately unimplemented.
- `worker/tools/check-listings.js` — the nightly sweep, oldest check first.
- System documentation section 8, *Where Listing Data Comes From*.

#### Changed

- Two deliberate asymmetries: an error outcome is never read as an absence, and a listing returning to market is acted on the **first** sighting because escrow fails.

## 0.6.0

2026-09-01

### The marketplace.

### Added

- Map-based browsing: filters across the top, map on the left, cards on the right, drill-down panel over it.
- `core.market_area`, `core.property_detail`, `core.property_media`, `core.saved_search`; views `api.property_detail`, `api.property_media`, `api.my_favorite`, `api.my_saved_search`.
- Favourites, saved searches, and a plain-English search box (a rules parser, not a model — the seam and its validator are what had to be right first).
- `web/media.js` — deterministic generated listing illustrations, so no stock service and nobody's copyright.
- 16 further demo listings spanning 1–6 beds, \$87k–\$581k, nine cities.
- System documentation section 7, *The Marketplace*.

### Security

- Photographs inherit the address gate. `property_media.reveals_location` marks exterior views, released on the same predicate as the street address — without it the gate would reopen through the picture gallery.
- Gated listings are drawn on the map as a fuzzed ring rather than a pin, because their coordinates are deliberately offset.

## 0.5.0

2026-09-01

### Authentication.

#### Added

- `core.credential` and `core.session`, both RLS-forced with no policy at all, reachable only through SECURITY DEFINER functions.
- `script` password hashing, session cookies, a login page.
- Account lockout after five failures; a password change revokes every session.

#### Changed

- Nothing beneath it. Not one policy, view or grant was modified to add authentication — the database contract was always "here is a role and an actor id", and a session now supplies those instead of a dropdown.
- The demo persona switcher is off unless `DEMO_PERSONAS=1`.

## 0.4.0

2026-08-31

### Deployable from published images.

#### Added

- `.github/workflows/release.yml` — publishes db, web and worker to GHCR.
- `docker-compose.release.yml` for pulling by tag; Portainer instructions.

#### Changed

- Role credentials are taken from the environment at first start rather than baked in. A role given no password stays NOLOGIN, failing at connect time rather than quietly granting access.

## 0.3.0

2026-08-31

### Review queue, deal pipeline, and the system documentation.

### Added

- `core.review_queue` and review actions with an allowlist on accepted CRM edits.
- Deal pipeline, stage history and per-party visibility.
- `docs/System-Documentation.pdf`, generated from the repository and a freshly built database rather than written from memory.

## 0.2.0

2026-08-31

### The GoHighLevel bridge.

#### Added

- Schema `ghl`: id mapping, webhook events, transactions, fee agreements, sync state, outbox.
- Integration worker: webhook intake with signature verification, outbox drain, document and transaction sync, EspoCRM migration passes.
- `docs/GHL-Interface-Specification.pdf`.

#### Security

- The fee gate requires **both** a completed document and a paid transaction. A signed-but-unpaid document does not open it.

## 0.1.0

2026-08-31

### Foundation: the visibility model.

#### Added

- PostgreSQL schema with three visibility bands enforced by row-level security and column grants, not by application code.
- Four-schema separation: `core` (tables, no app-role access), `sec` (predicates), `api` (masking views), plus the roles that use them.
- `api.security_invariants()` — must always return zero rows.
- Demo dataset and the Marcus/Ruth pair: same role, same query, one timestamp of difference.